

Towards End-to-End Optimization

of LLM-based Applications with Ayo

Yihang Zhu, Yiqun Wang

3/26/2026

The University of Michigan

Table of Contents

1. Background & Motivation
2. Ayo Framework: Overall View
3. Graph Optimization & Scheduling
4. Evaluation & Conclusion

Background & Motivation

Context: LLM-based Applications

Complex Workflows:

- Tight integration of LLMs with non-LLM components (Search, Vector DBs, APIs).
- Two inference phases: **Prefilling** (compute) & **Decoding** (memory).

The Orchestration Bottleneck:

- Current frameworks use coarse-grained, sequential module execution.

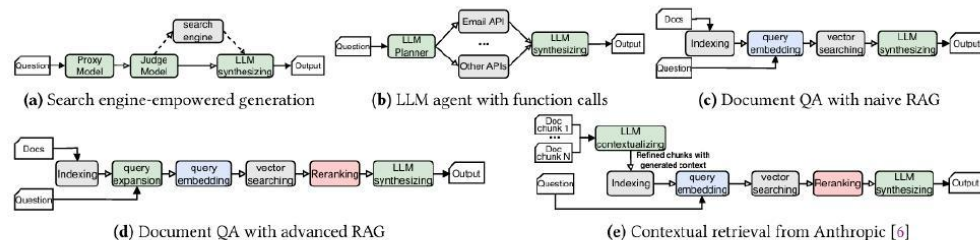


Figure 2: Common real-world LLM-based application workflows.

Context: LLM-based Applications

Complex Workflows:

- Tight integration of LLMs with non-LLM components (Search, Vector DBs, APIs).
- Two inference phases:
Prefilling (compute) &
Decoding (memory).

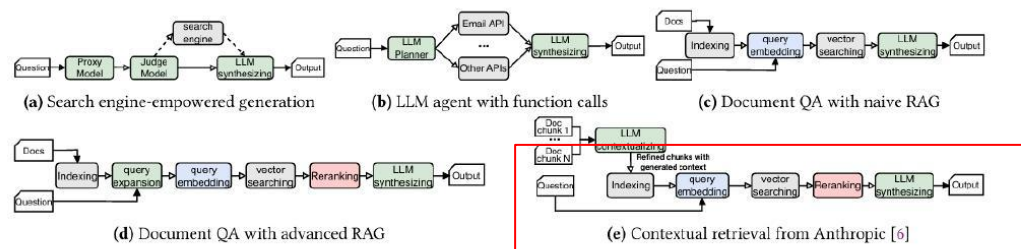
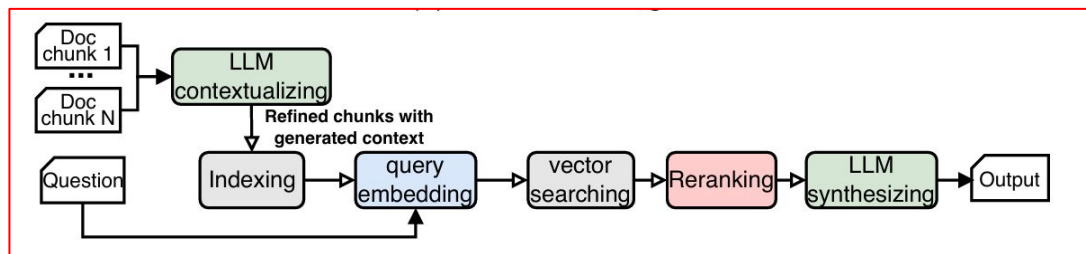


Figure 2: Common real-world LLM-based application workflows.



Motivation: Prevalence & Latency Breakdown

Widespread Adoption

Complex LLM workflows heavily dominate real-world applications.

Workflow	Count	Proportion
SE generation	16	53.33%
LLM agent	13	43.33%
Doc QA w/ naive RAG	26	86.67%
Doc QA w/ advanced RAG	23	76.67%

Table 1: Workflow occurrence in 30 sampled GitHub projects.

End-to-End Latency Matters

Non-LLM modules heavily dominate the critical path.

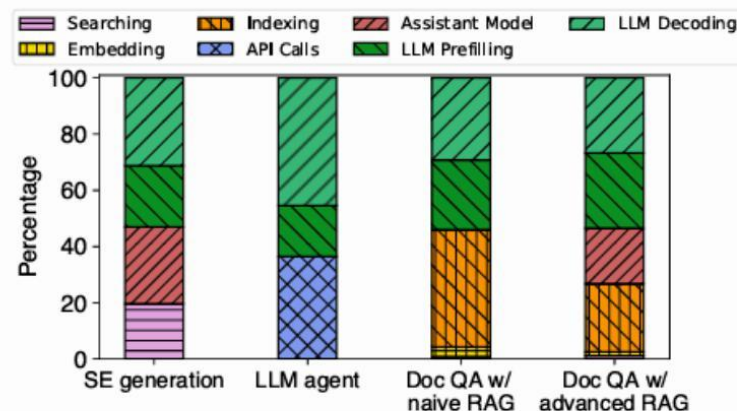


Figure 1: Non-LLM modules account for > 50% of latency.

Context: LLM-based Applications

The Orchestration Bottleneck:

- Current frameworks use coarse-grained, sequential module execution.

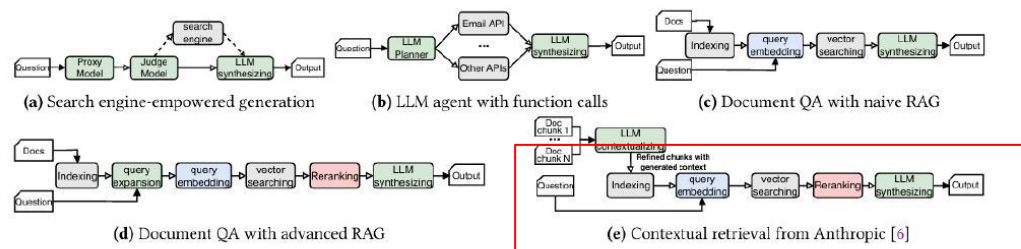
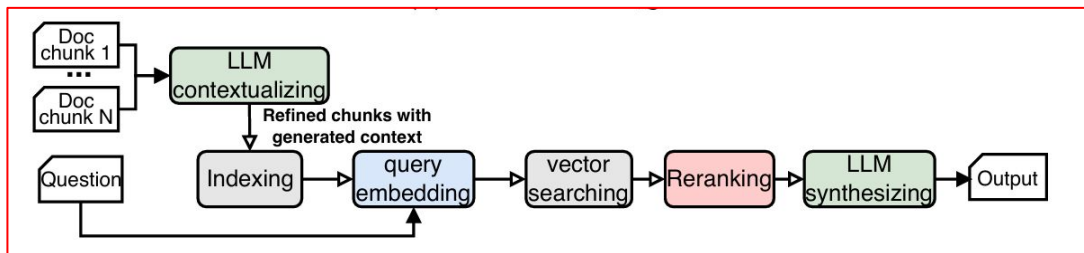


Figure 2: Common real-world LLM-based application workflows.



Motivation: The Black Box Bottleneck

Missed Optimization Opportunities

Coarse-grained execution creates sequential barriers across black-box modules, preventing cross-module parallelization and yielding suboptimal backend scheduling.

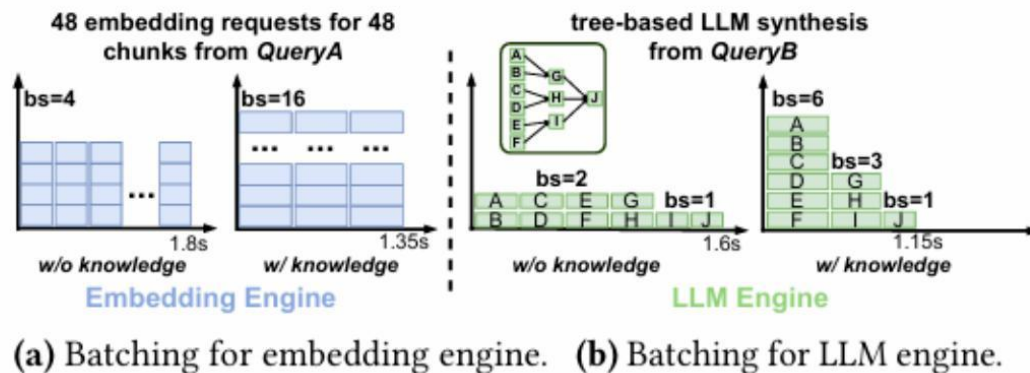
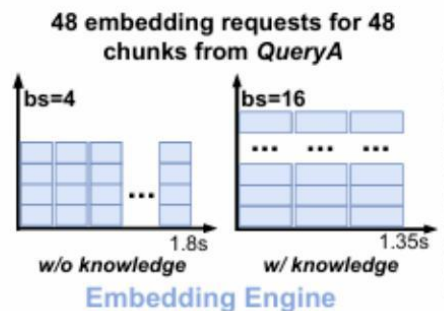


Figure 4: Scheduling inefficiencies: Conventional request-level scheduling struggles with request dependencies across primitives (e.g., tree-based synthesis).

Motivation: The Black Box Bottleneck

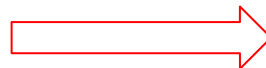
Missed Optimization Opportunities

Coarse-grained execution creates sequential barriers across black-box modules, preventing cross-module parallelization and yielding suboptimal backend scheduling.



(a) Batching for embedding engine

48 Requests



module

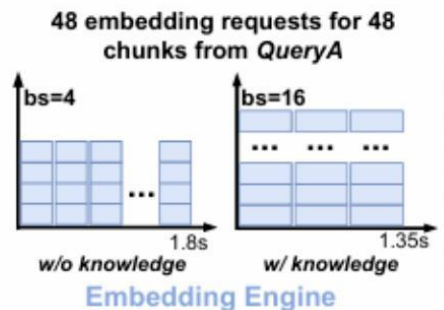
No knowledge: use **small** batch size.
Large batch size leads to higher latency if requests come from different query.

Figure 4: Scheduling inefficiencies: Conventional request-level scheduling struggles with request dependencies across primitives (e.g., tree-based synthesis).

Motivation: The Black Box Bottleneck

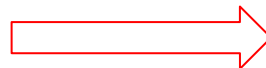
Missed Optimization Opportunities

Coarse-grained execution creates sequential barriers across black-box modules, preventing cross-module parallelization and yielding suboptimal backend scheduling.



(a) Batching for embedding engine

48 Requests



module

With knowledge: use **large** batch size.
Handle only part of the query is meaningless.
Process as fast as possible.

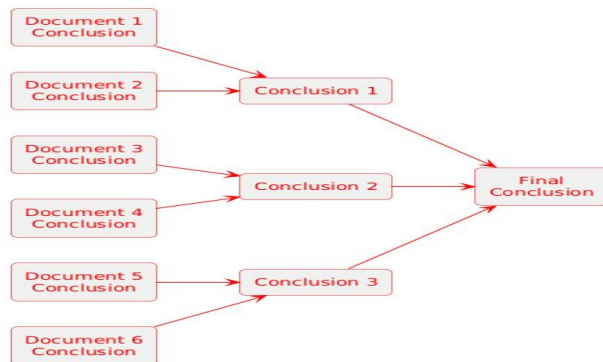
Request Correlation

Figure 4: Scheduling inefficiencies: Conventional request-level scheduling struggles with request dependencies across primitives (e.g., tree-based synthesis).

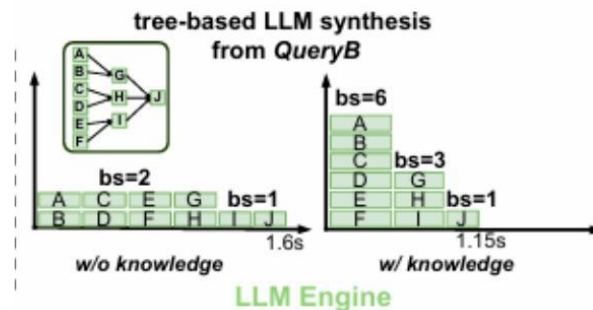
Motivation: The Black Box Bottleneck

Missed Optimization Opportunities

Coarse-grained execution creates sequential barriers across black-box modules, preventing cross-module parallelization and yielding suboptimal backend scheduling.



No knowledge: use **small** batch size.
Treat each request independently.



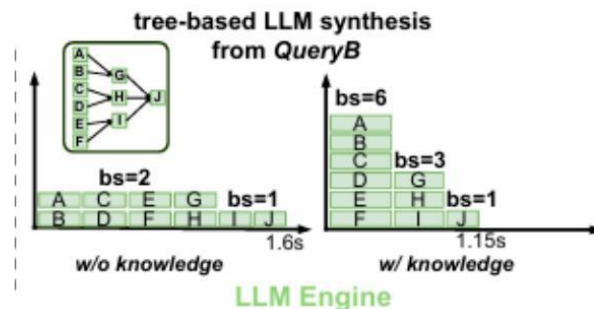
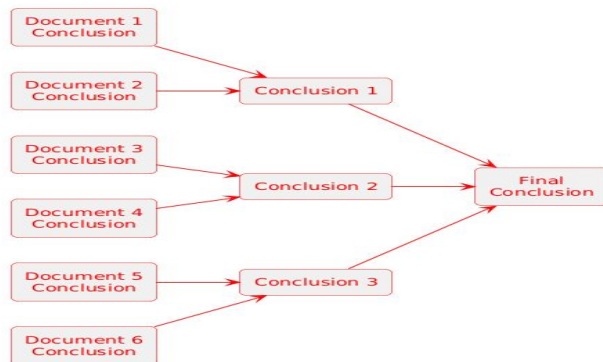
engine. (b) Batching for LLM engine.

Figure 4: Scheduling inefficiencies: Conventional request-level scheduling struggles with request dependencies across primitives (e.g., tree-based synthesis).

Motivation: The Black Box Bottleneck

Missed Optimization Opportunities

Coarse-grained execution creates sequential barriers across black-box modules, preventing cross-module parallelization and yielding suboptimal backend scheduling.



With knowledge: use **adaptive** batch size.
Handle each layer with larger batch size.
Lower end to end latency.

Request Dependency

Figure 4: Scheduling inefficiencies: Conventional request-level scheduling struggles with request dependencies across primitives (e.g., tree-based synthesis).

Ayo Framework: Overall View

Paradigm Shift: Breaking the Black Box

From Modules to Primitives

Ayo replaces coarse-grained modules with fine-grained orchestration, unlocking parallelization and pipelining across the entire workflow.

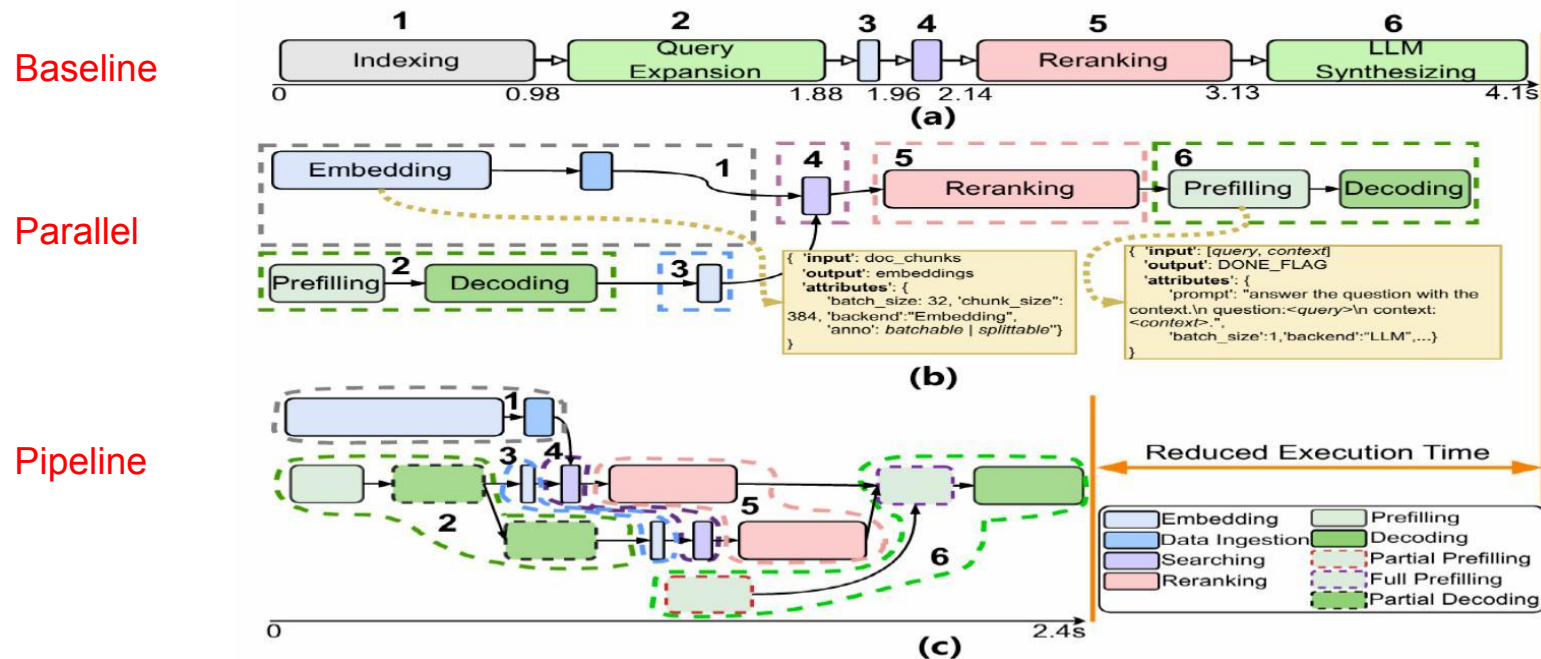


Figure 3: (a) Baseline vs. Ayo's (b) parallel & (c) pipeline.

Paradigm Shift: Breaking the Black Box

From Modules to Primitives

Ayo replaces coarse-grained modules with fine-grained orchestration, unlocking parallelization and pipelining across the entire workflow. **Large & sequential modules**

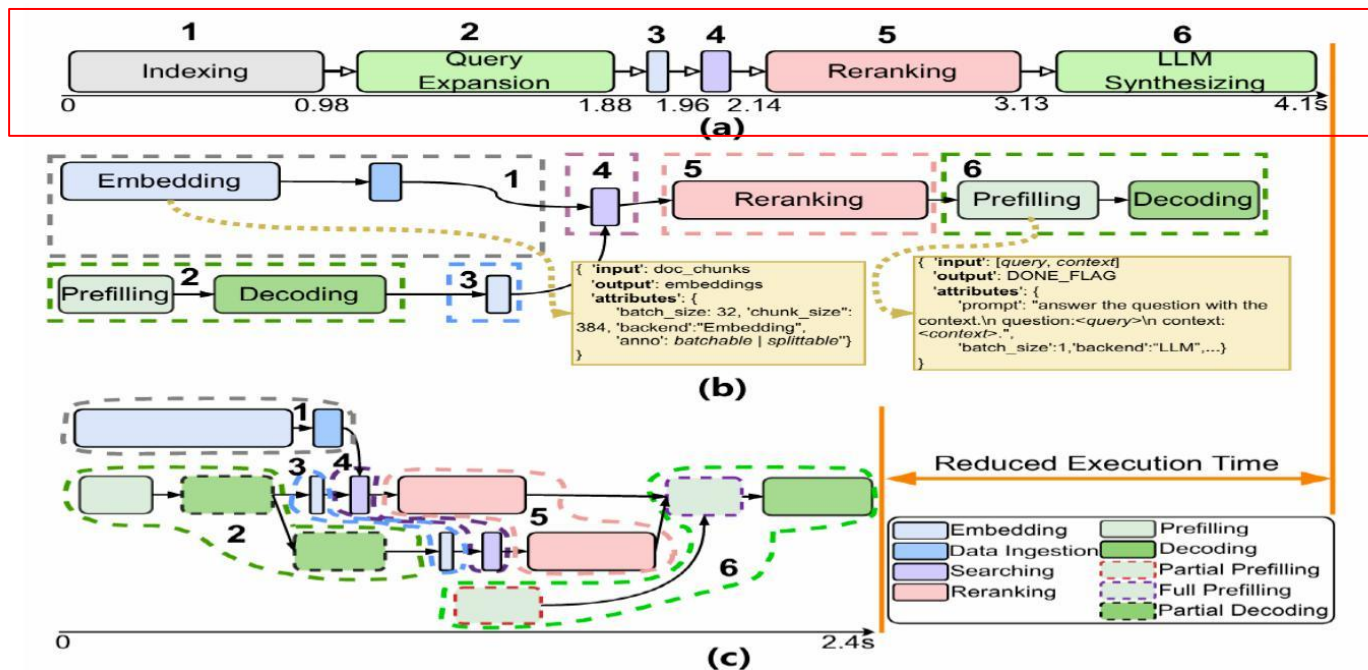


Figure 3: (a) Baseline vs. Ayo's (b) parallel & (c) pipeline.

Paradigm Shift: Breaking the Black Box

From Modules to Primitives

Ayo replaces coarse-grained modules with fine-grained orchestration, unlocking parallelization and pipelining across the entire workflow.

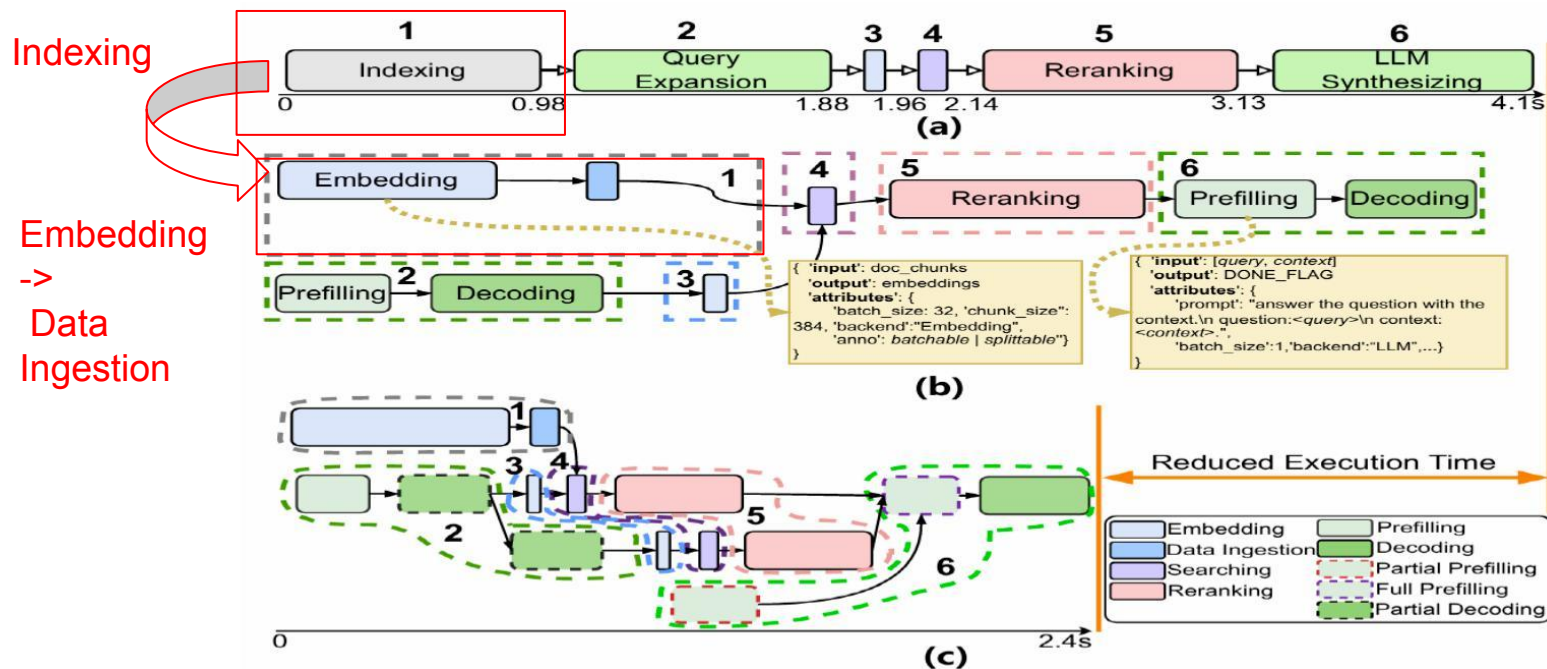


Figure 3: (a) Baseline vs. Ayo's (b) parallel & (c) pipeline.

Paradigm Shift: Breaking the Black Box

From Modules to Primitives

Ayo replaces coarse-grained modules with fine-grained orchestration, unlocking parallelization and pipelining across the entire workflow.

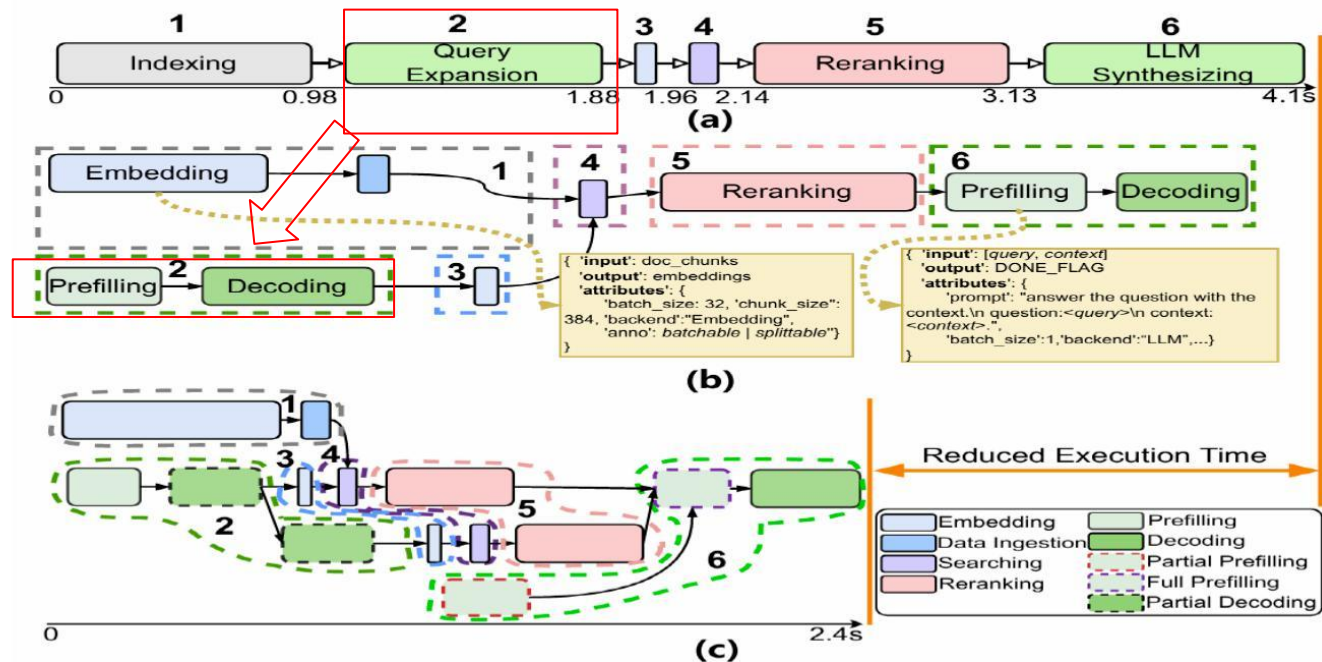


Figure 3: (a) Baseline vs. Ayo's (b) parallel & (c) pipeline.

Paradigm Shift: Breaking the Black Box

From Modules to Primitives

Ayo replaces coarse-grained modules with fine-grained orchestration, unlocking parallelization and pipelining across the entire workflow.

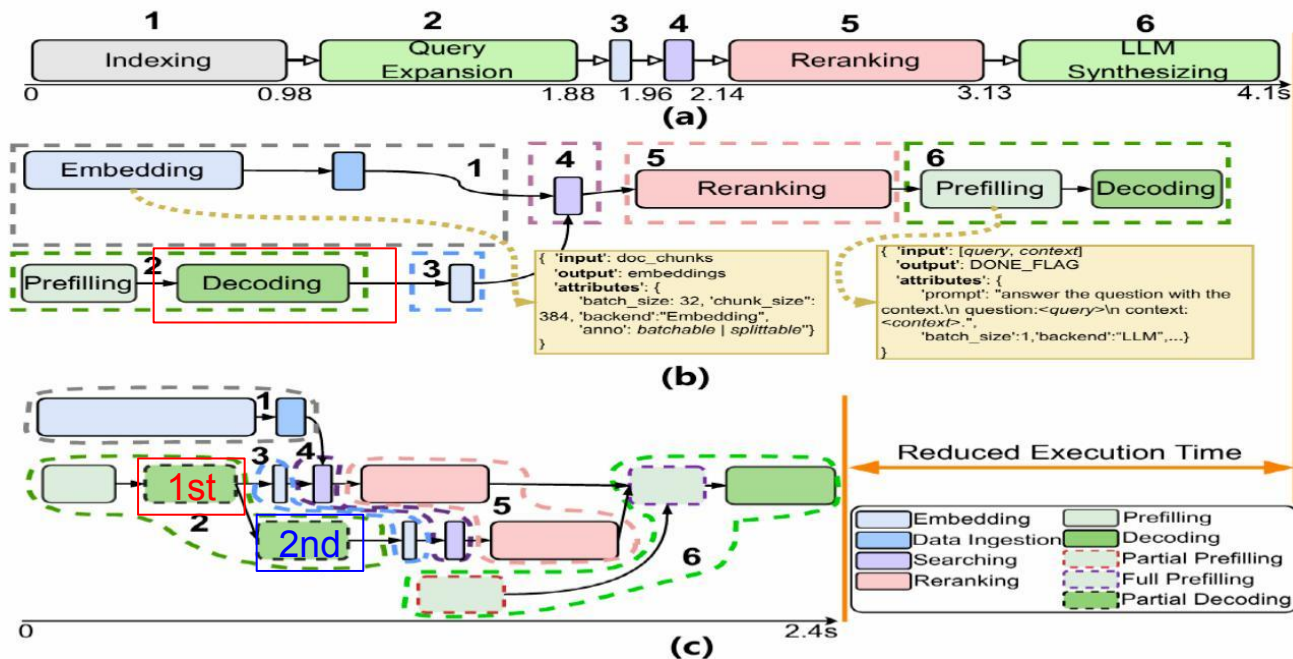


Figure 3: (a) Baseline vs. Ayo's (b) parallel & (c) pipeline.

Paradigm Shift: Breaking the Black Box

From Modules to Primitives

Ayo replaces coarse-grained modules with fine-grained orchestration, unlocking parallelization and pipelining across the entire workflow.

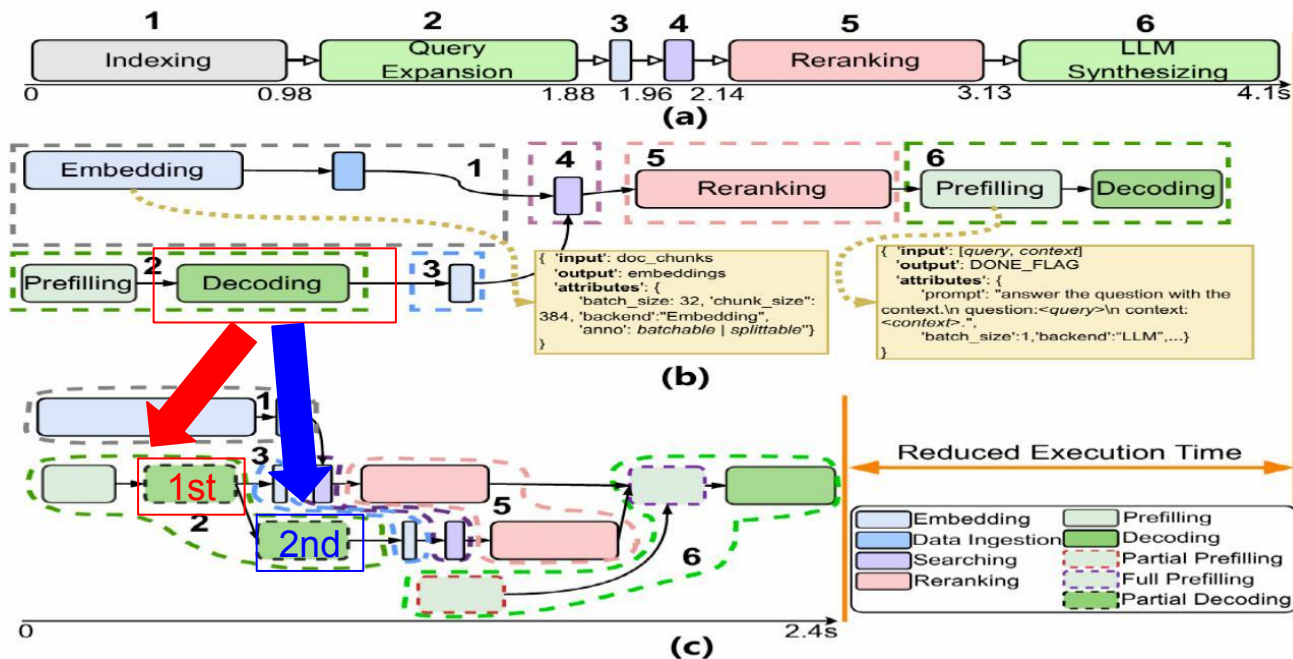


Figure 3: (a) Baseline vs. Ayo's (b) parallel & (c) pipeline.

Paradigm Shift: Breaking the Black Box

From Modules to Primitives

Ayo replaces coarse-grained modules with fine-grained orchestration, unlocking parallelization and pipelining across the entire workflow.

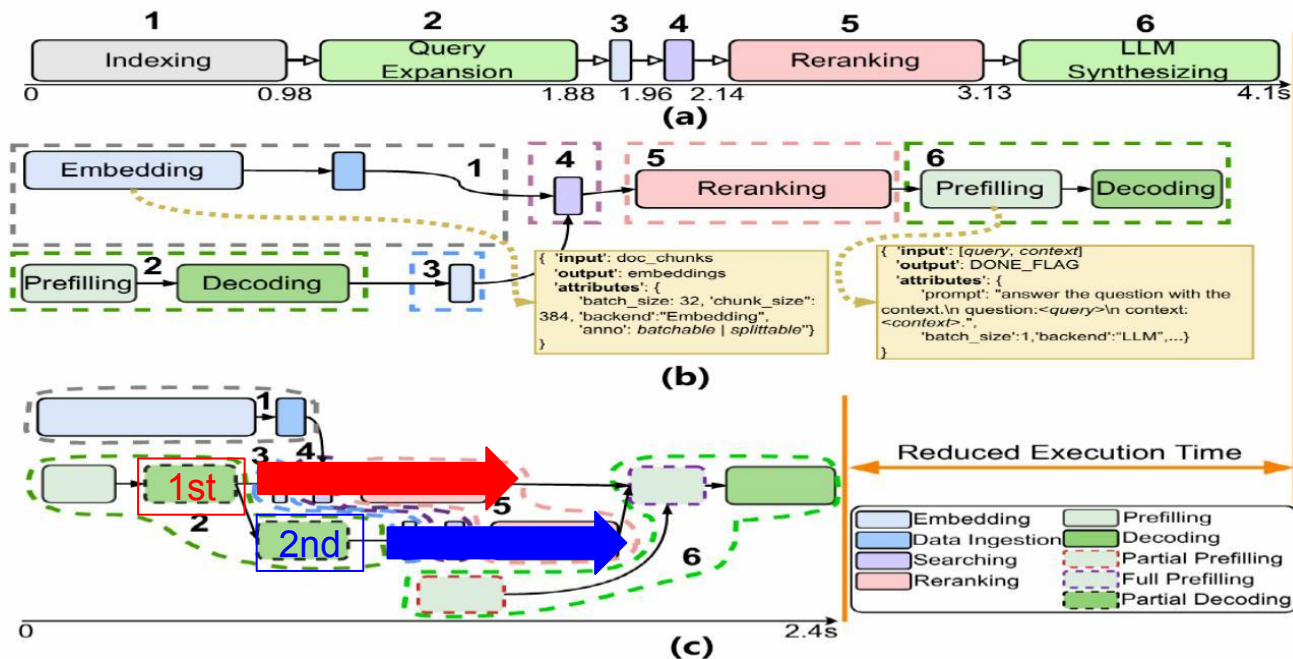


Figure 3: (a) Baseline vs. Ayo's (b) parallel & (c) pipeline.

Defining Task Primitives

The Primitive-level Dataflow Graph (p-Graph):

- Ayo decomposes large functional modules into the smallest indivisible execution units (Primitives).
- Explicitly exposes internal execution states and data dependencies.

Type	Description
Reranking	Compute and rank the relevance scores for the query and context pairs.
Ingestion	Store embedding vectors into vector database
Searching	Perform vector searching in the database
Embedding	Create embedding vectors for docs or questions
Prefilling	The prefilling part of LLM inference
Decoding	The decoding part of LLM inference
Partial Prefilling	Prefilling for partial prefix of a prompt (e.g. instruction, context, question)
Full Prefilling	Prefilling for rest part of a prompt after a partial prefilling
Partial Decoding	Part of full decoding for partial output
Condition	Decide the conditional branch
Aggregate	Aggregate the results from multiple primitives

Table 2: Representative task primitives defined in the Ayo framework.

Defining Task Primitives

The Primitive-level Dataflow Graph (p-Graph):

- Ayo decomposes large functional modules into the smallest indivisible execution units (Primitives).
- Explicitly exposes internal execution states and data dependencies.

Non-LLM primitives

Type	Description
Reranking	Compute and rank the relevance scores for the query and context pairs.
Ingestion	Store embedding vectors into vector database
Searching	Perform vector searching in the database
Embedding	Create embedding vectors for docs or questions
Prefilling	The prefilling part of LLM inference
Decoding	The decoding part of LLM inference
Partial Prefilling	Prefilling for partial prefix of a prompt (e.g. instruction, context, question)
Full Prefilling	Prefilling for rest part of a prompt after a partial prefilling
Partial Decoding	Part of full decoding for partial output
Condition	Decide the conditional branch
Aggregate	Aggregate the results from multiple primitives

Table 2: Representative task primitives defined in the Ayo framework.

Defining Task Primitives

The Primitive-level Dataflow Graph (p-Graph):

- Ayo decomposes large functional modules into the smallest indivisible execution units (Primitives).
- Explicitly exposes internal execution states and data dependencies.

Type	Description
Reranking	Compute and rank the relevance scores for the query and context pairs.
Ingestion	Store embedding vectors into vector database
Searching	Perform vector searching in the database
Embedding	Create embedding vectors for docs or questions
Prefilling	The prefilling part of LLM inference
Decoding	The decoding part of LLM inference
Partial Prefilling	Prefilling for partial prefix of a prompt (e.g. instruction, context, question)
Full Prefilling	Prefilling for rest part of a prompt after a partial prefilling
Partial Decoding	Part of full decoding for partial output
Condition	Decide the conditional branch
Aggregate	Aggregate the results from multiple primitives

LLM primitives

Table 2: Representative task primitives defined in the Ayo framework.

Defining Task Primitives

The Primitive-level Dataflow Graph (p-Graph):

- Ayo decomposes large functional modules into the smallest indivisible execution units (Primitives).
- Explicitly exposes internal execution states and data dependencies.

Type	Description
Reranking	Compute and rank the relevance scores for the query and context pairs.
Ingestion	Store embedding vectors into vector database
Searching	Perform vector searching in the database
Embedding	Create embedding vectors for docs or questions
Prefilling	The prefilling part of LLM inference
Decoding	The decoding part of LLM inference
Partial Prefilling	Prefilling for partial prefix of a prompt (e.g. instruction, context, question)
Full Prefilling	Prefilling for rest part of a prompt after a partial prefilling
Partial Decoding	Part of full decoding for partial output
Condition	Decide the conditional branch
Aggregate	Aggregate the results from multiple primitives

Branching & Sync

Table 2: Representative task primitives defined in the Ayo framework.

Ayo System Architecture

1. Offline Profiling

Builds a Cost Model via engine probing for pipelining decisions.

2. Frontend Optimizer

Decomposes queries into p-Graphs and applies AOT optimizations (parallelization & pipelining).

3. Backend Scheduler

Dispatches tasks based on *topological depth*, prioritizing critical bottlenecks.

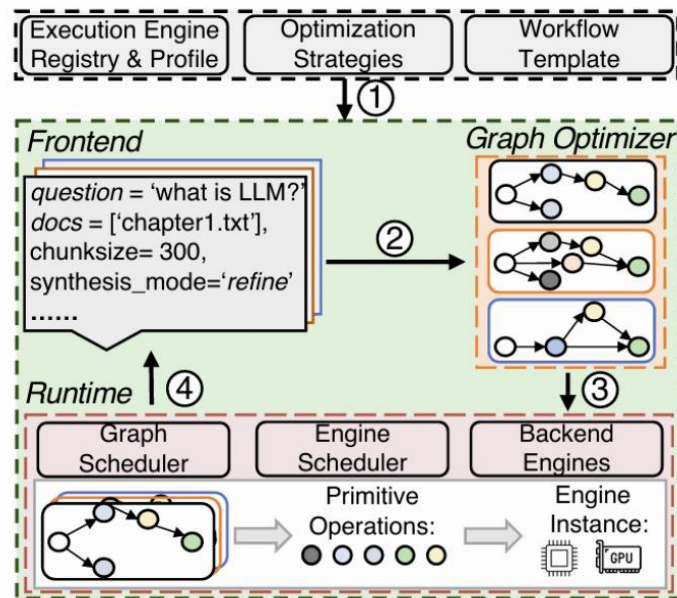


Figure 5: The comprehensive system architecture of Ayo.

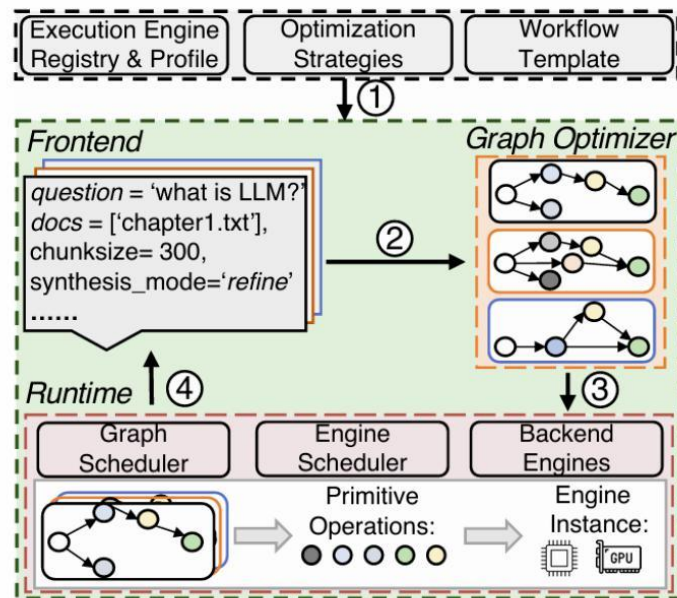


Figure 5: The comprehensive system architecture of Ayo.

Ayo System Architecture

1. Offline Profiling

Builds a Cost Model via engine probing for pipelining decisions.

Which engines do you have?
What are the latency of them under diff. workloads?
What does the template look like?
How do you want to optimize?

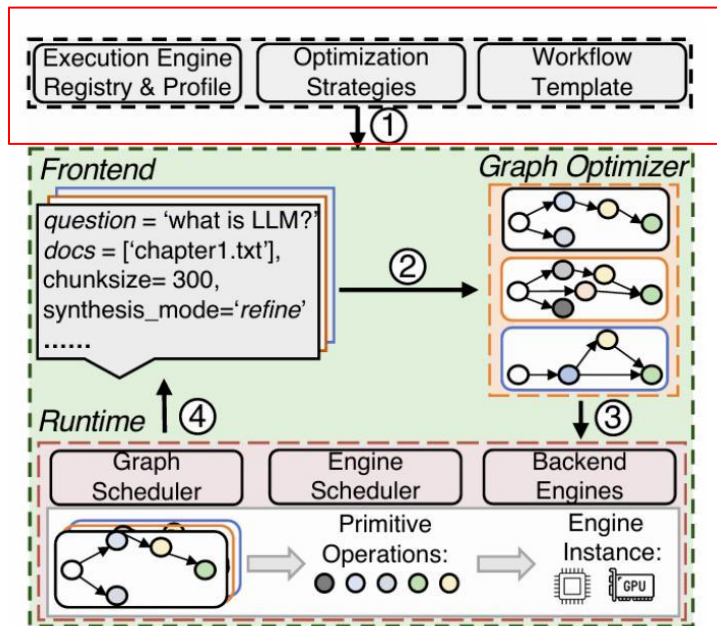


Figure 5: The comprehensive system architecture of Ayo.

Ayo System Architecture

1. Offline Profiling

Builds a Cost Model via engine probing for pipelining decisions.

2. Frontend Optimizer

Decomposes queries into p-Graphs and applies AOT optimizations (parallelization & pipelining).

Given input,
build the graph,
optimize it

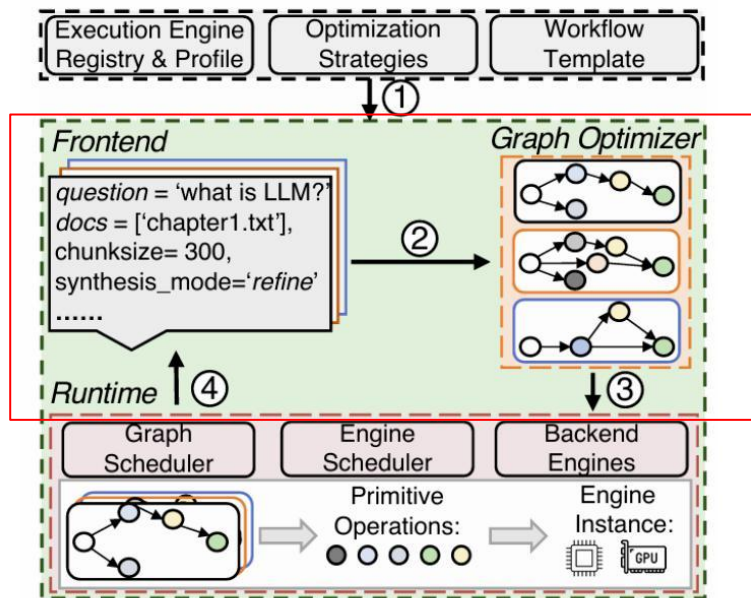


Figure 5: The comprehensive system architecture of Ayo.

Ayo System Architecture

1. Offline Profiling

Builds a Cost Model via engine probing for pipelining decisions.

2. Frontend Optimizer

Decomposes queries into p-Graphs and applies AOT optimizations (parallelization & pipelining).

3. Backend Scheduler

Dispatches tasks based on *topological depth*, prioritizing critical bottlenecks.

Which primitives are ready?

How to run ready primitives efficiently?

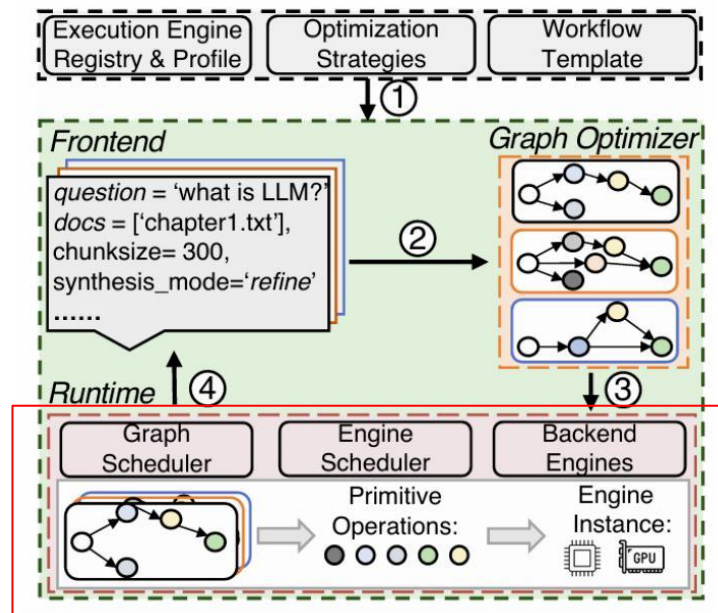


Figure 5: The comprehensive system architecture of Ayo.

Graph Optimization & Scheduling

Execution graph (e-Graph) Optimization

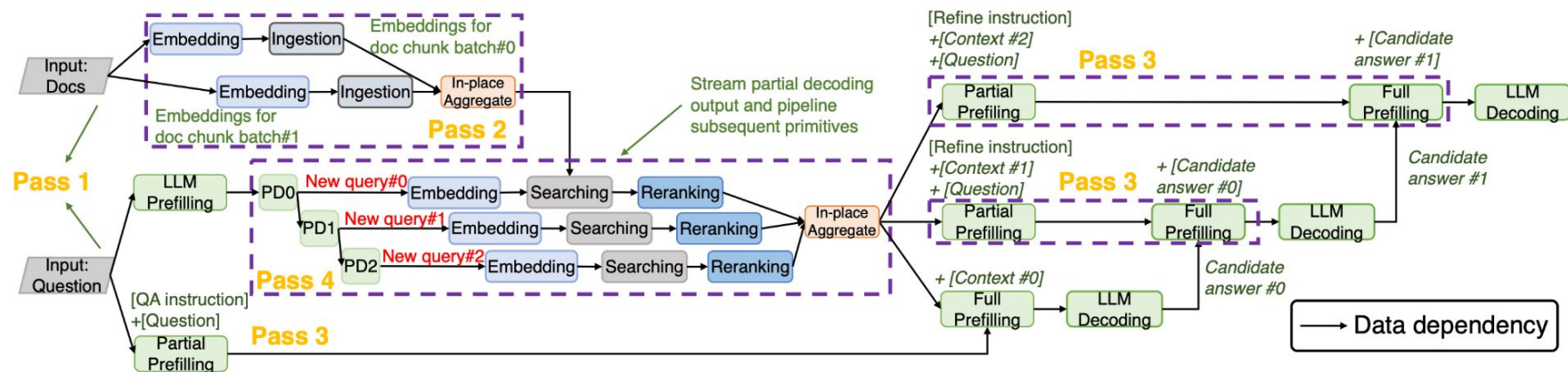
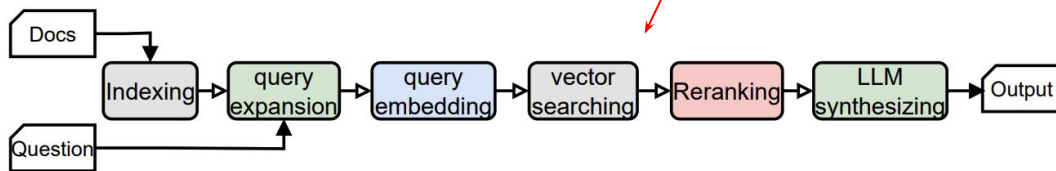


Figure 6. Illustrative optimized e-graph of a query for advanced RAG-based document QA with a *refine* synthesis mode. (PD: partial decoding; annotated computed prompt part for Partial/Full Prefilling; primitive metadata omitted; block length not indicative of execution time.)



(d) Document QA with advanced RAG

Execution graph (e-Graph) Optimization

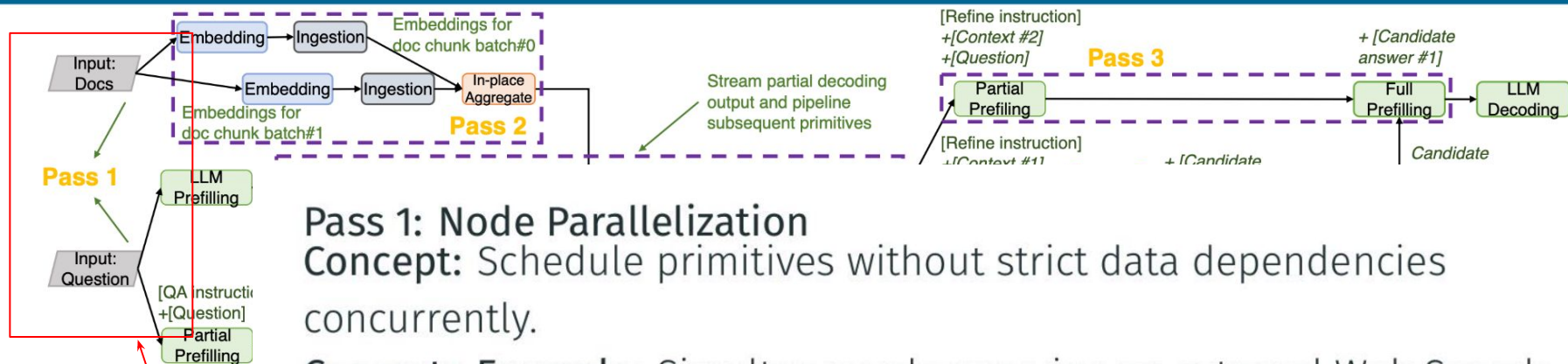
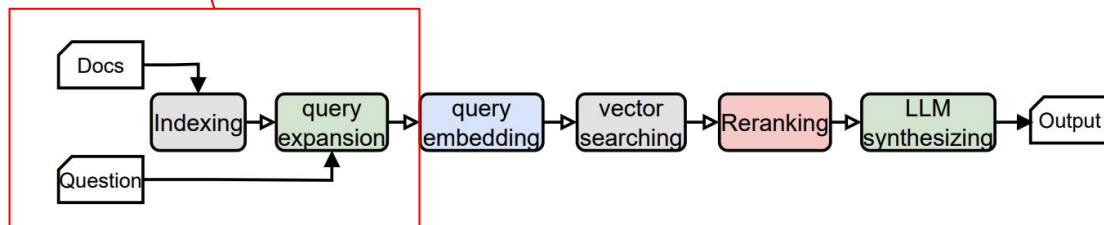


Figure 6. Illustrative annotated computation graph

Pass 1: Node Parallelization

Concept: Schedule primitives without strict data dependencies concurrently.

Concrete Example: Simultaneously querying an external Web Search API and retrieving local documents from a Vector DB, rather than doing them sequentially.



(d) Document QA with advanced RAG

Execution graph (e-Graph) Optimization

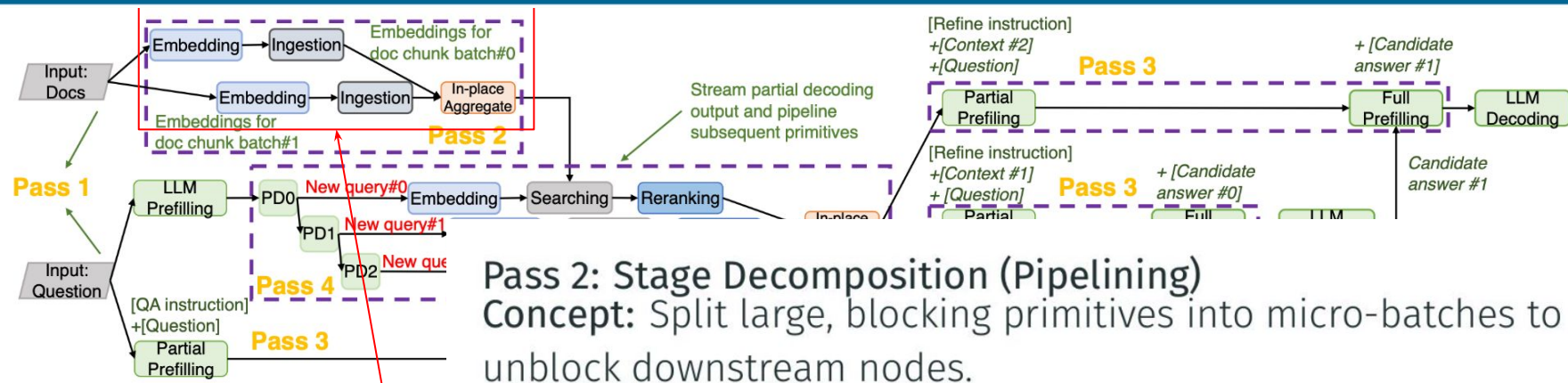
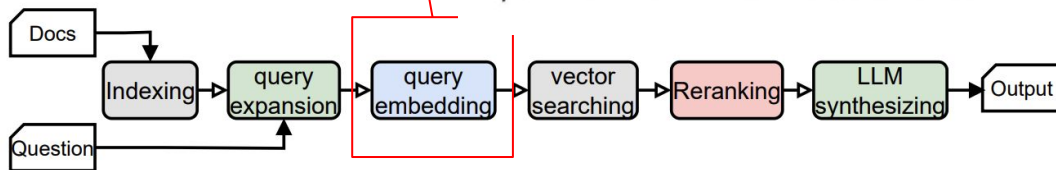


Figure 6. Illustrative optimized e-graph annotated computed prompt part for Pass 2

Pass 2: Stage Decomposition (Pipelining)

Concept: Split large, blocking primitives into micro-batches to unblock downstream nodes.

Concrete Example: When embedding 1,000 document chunks, Ayo splits them into batches of 100. Once the first 100 are embedded, they are immediately passed to the Reranker, while the embedder processes the next batch.



(d) Document QA with advanced RAG

Execution graph (e-Graph) Optimization

Pass 3: Causal Prefilling

Concept: Overlap static instruction prefilling with upstream I/O tasks.

Concrete Example: While the Retriever is busy fetching context from the database, preemptively send the fixed system prompt ("You are a helpful AI...") to the LLM to compute and store in the KV Cache.

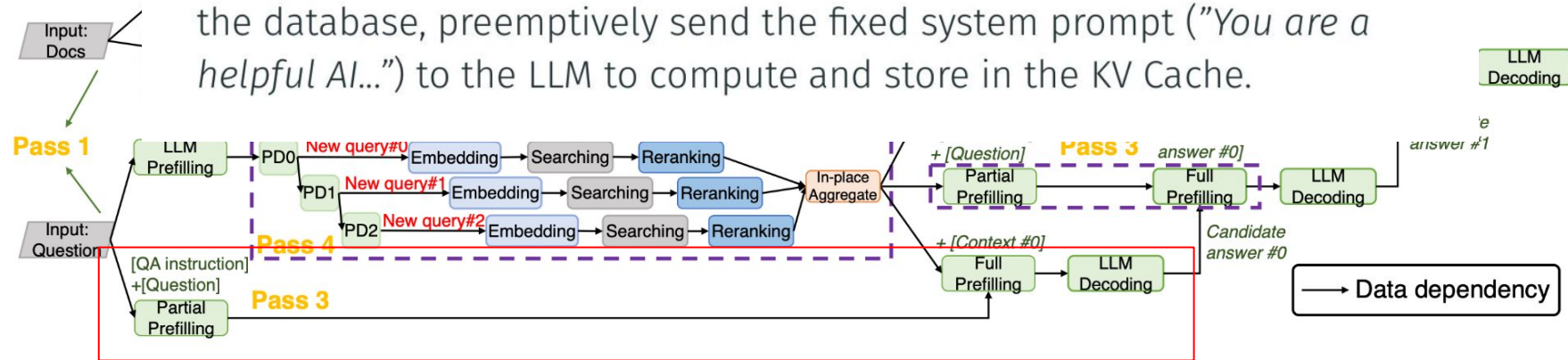
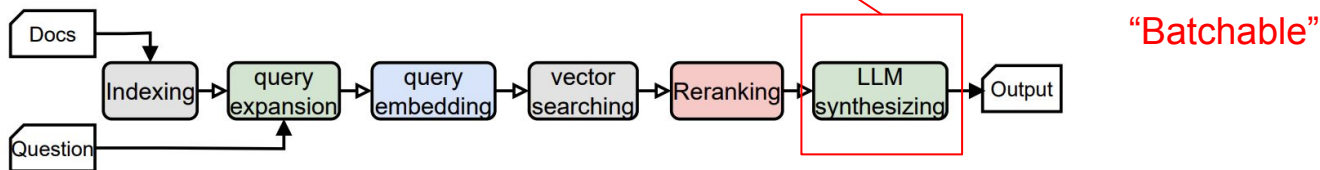


Figure 6. Illustrative optimized e-graph of a query for advanced RAG-based document QA with a *refine* synthesis mode. (PD: partial decoding; annotated computed prompt part for Partial/Full Prefilling; primitive metadata omitted; block length not indicative of execution time.)



(d) Document QA with advanced RAG

Execution graph (e-Graph) Optimization

Pass 4: Decoding Pipelining

Concept: Stream structural chunks from autoregressive decoding directly to downstream components.

Concrete Example: Instead of waiting for the LLM to generate an entire JSON object, stream parsed parameters directly to an external API the moment they are decoded.

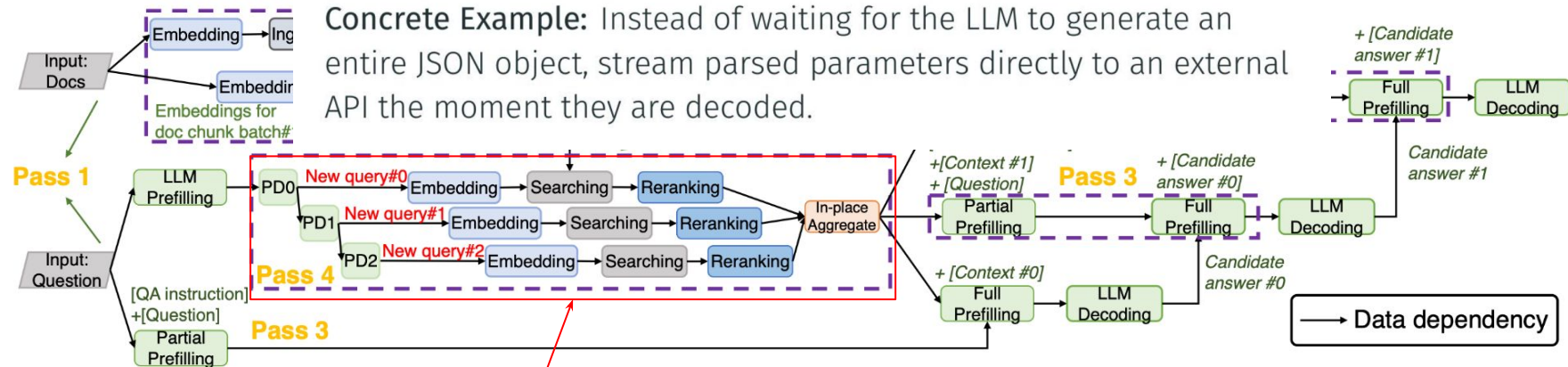
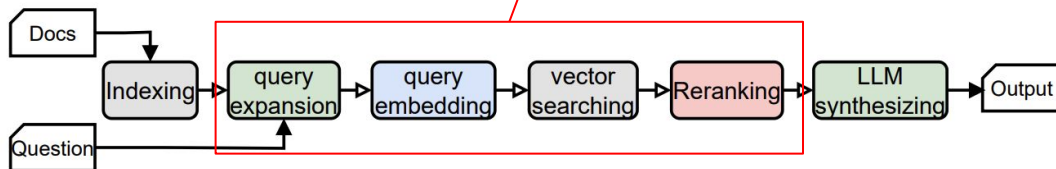


Figure 6. Illustrative optimized e-graph of a query for advanced RAG-based document QA with a *refine* synthesis mode. (PD: partial decoding; annotated computed prompt part for Partial/Full Prefilling; primitive metadata omitted; block length not indicative of execution time.)



(d) Document QA with advanced RAG

Frontend Optimizations: General Workflow

The Graph Optimizer transforms the p-Graph through targeted passes.

Pass 1: Node Parallelization

Concept: Schedule primitives without strict data dependencies concurrently.

Concrete Example: Simultaneously querying an external Web Search API and retrieving local documents from a Vector DB, rather than doing them sequentially.

Pass 2: Stage Decomposition (Pipelining)

Concept: Split large, blocking primitives into micro-batches to unblock downstream nodes.

Concrete Example: When embedding 1,000 document chunks, Ayo splits them into batches of 100. Once the first 100 are embedded, they are immediately passed to the Reranker, while the embedder processes the next batch.

Pass 3: Causal Prefilling

Concept: Overlap static instruction prefilling with upstream I/O tasks.

Concrete Example: While the Retriever is busy fetching context from the database, preemptively send the fixed system prompt (*"You are a helpful AI..."*) to the LLM to compute and store in the KV Cache.

Pass 4: Decoding Pipelining

Concept: Stream structural chunks from autoregressive decoding directly to downstream components.

Concrete Example: Instead of waiting for the LLM to generate an entire JSON object, stream parsed parameters directly to an external API the moment they are decoded.

Case Study: e-Graph for Advanced RAG Workflow

Synergistic Application of Optimization Passes:

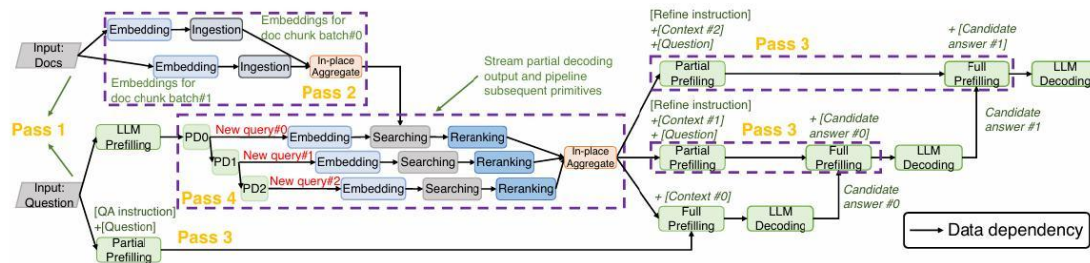
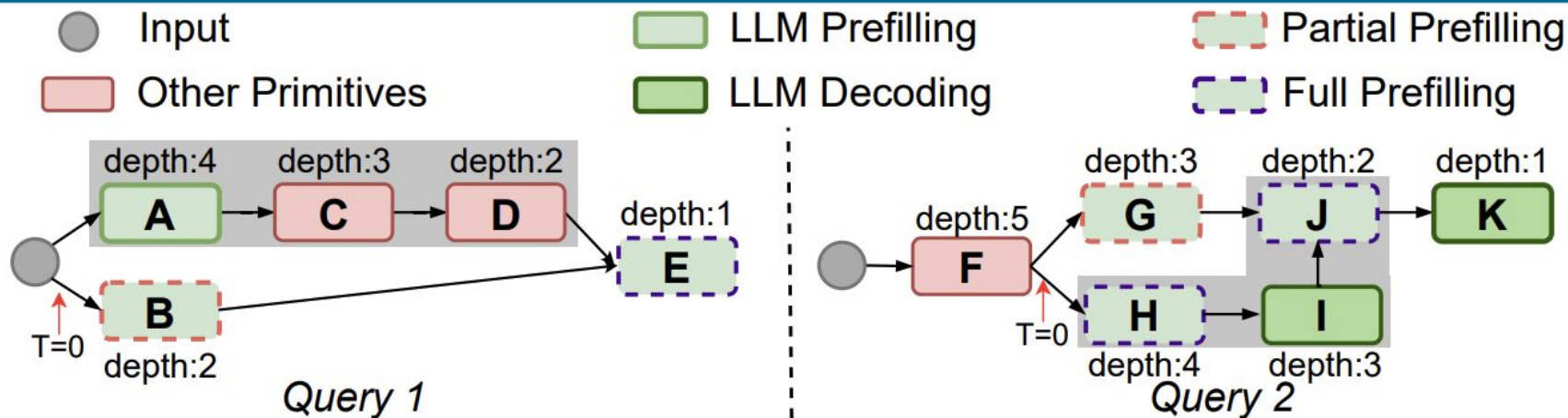


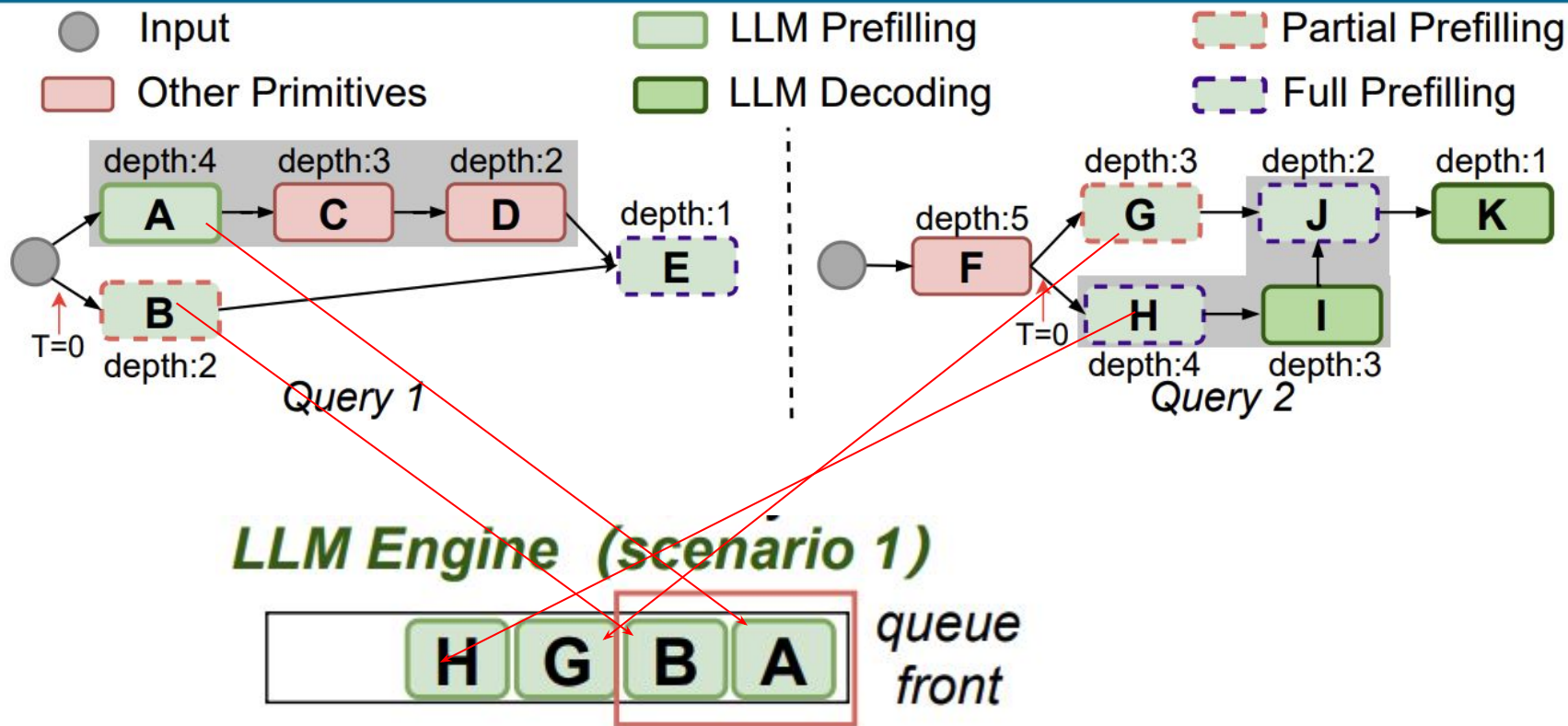
Figure 5: Optimized execution graph (e-Graph) demonstrating full workflow optimization.

- **Pass 4 (Decoding Pipelining):** Streams partially decoded expanded queries immediately to downstream retrieval modules.
- **Pass 1 (Node Parallelization):** Forks the retrieval process into concurrent branches (Embedding → Searching → Reranking).
- **Pass 2 & 3:** Pipelined micro-batch data ingestion, computing KV cache for static instructions concurrently with retrieval.

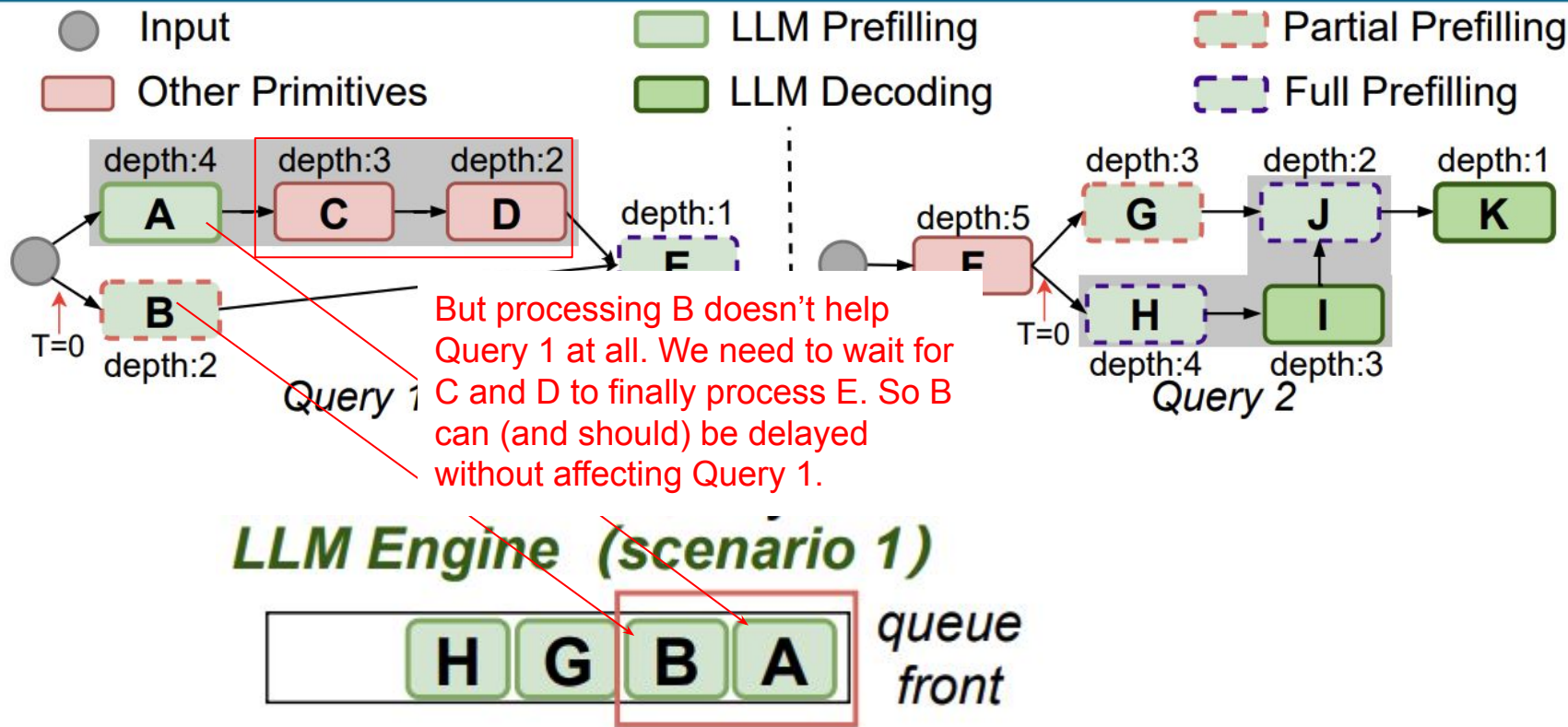
Runtime Scheduling: Topology-aware batching



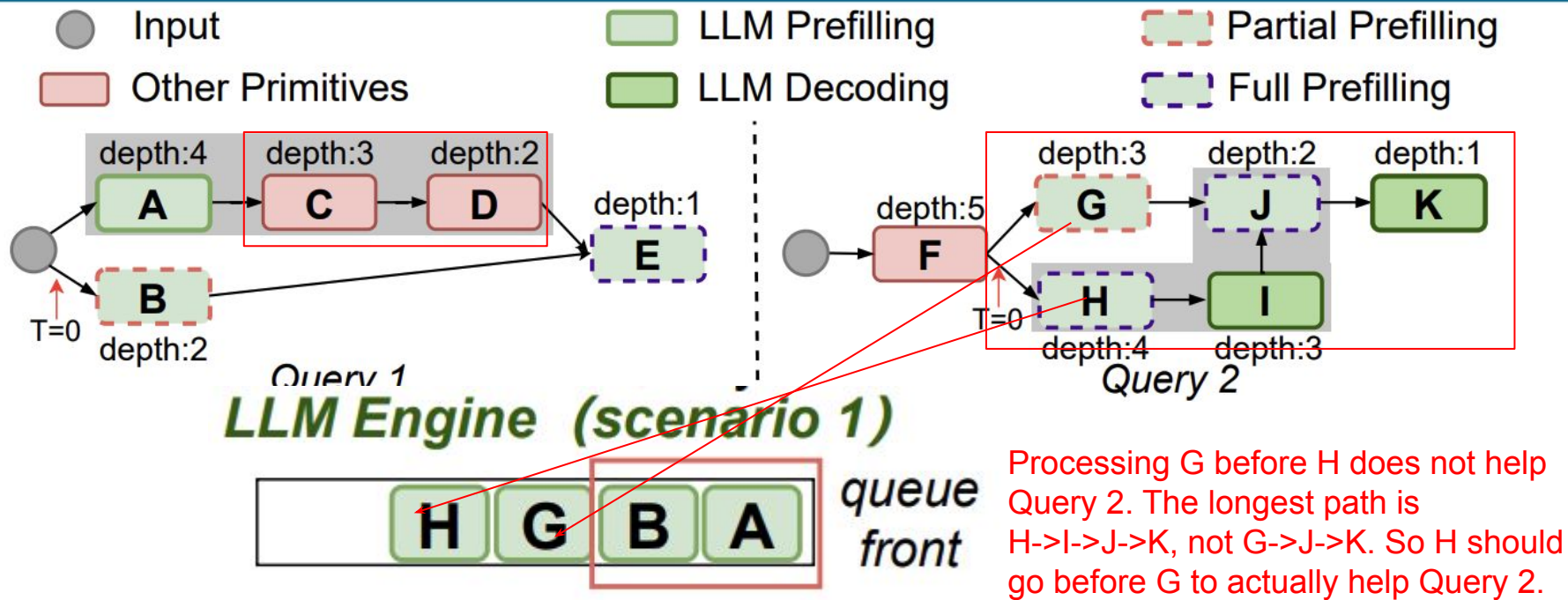
Runtime Scheduling: Topology-aware batching



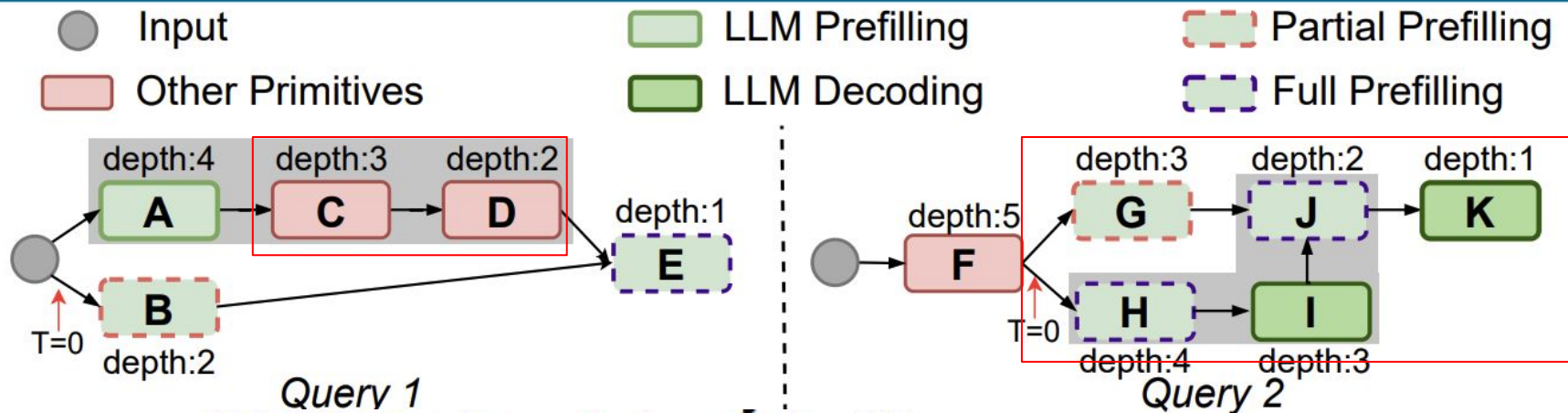
Runtime Scheduling: Topology-aware batching



Runtime Scheduling: Topology-aware batching



Runtime Scheduling: Topology-aware batching



LLM Engine (cenário 1)

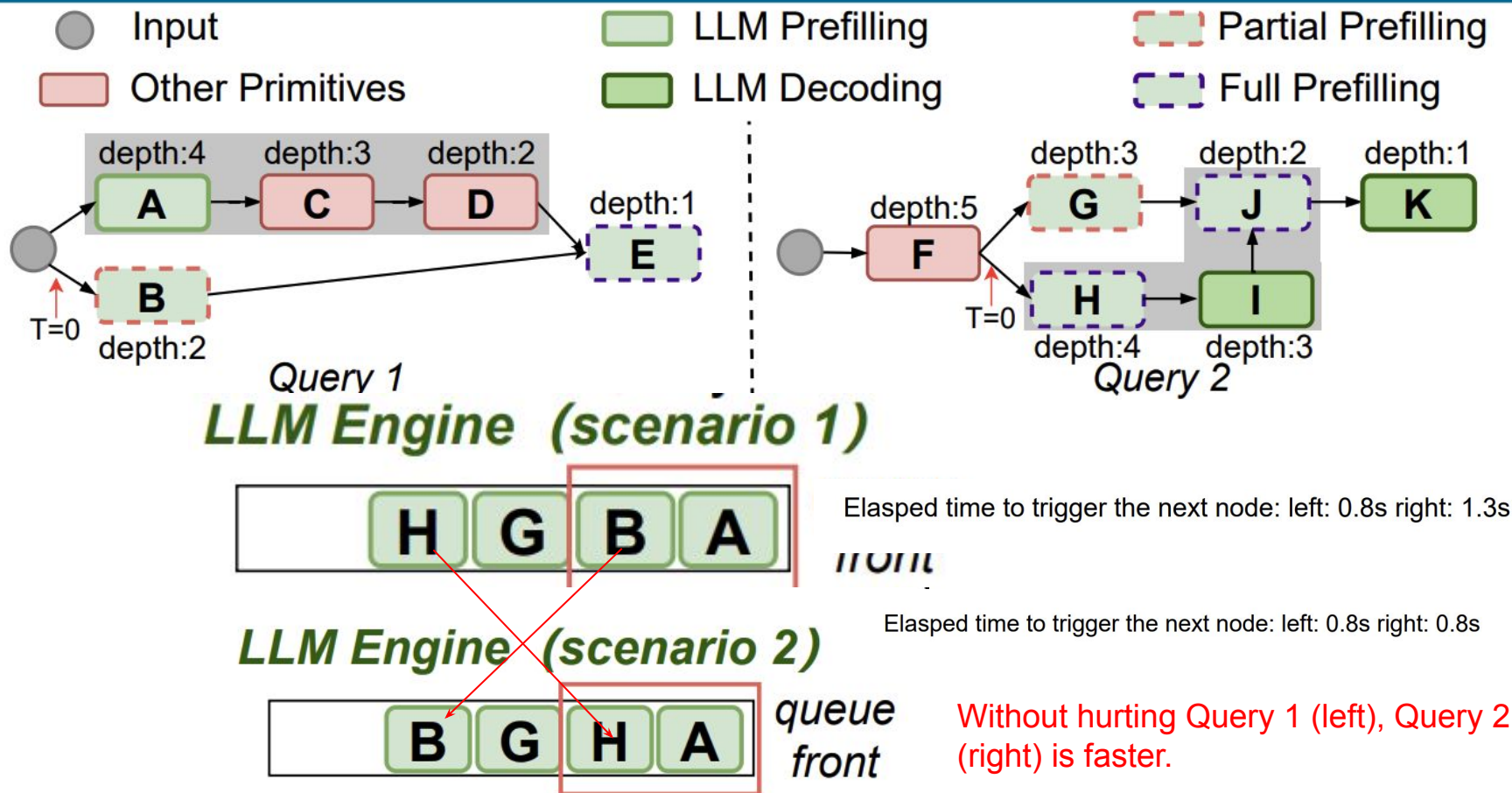


LLM Engine (scenario 2)

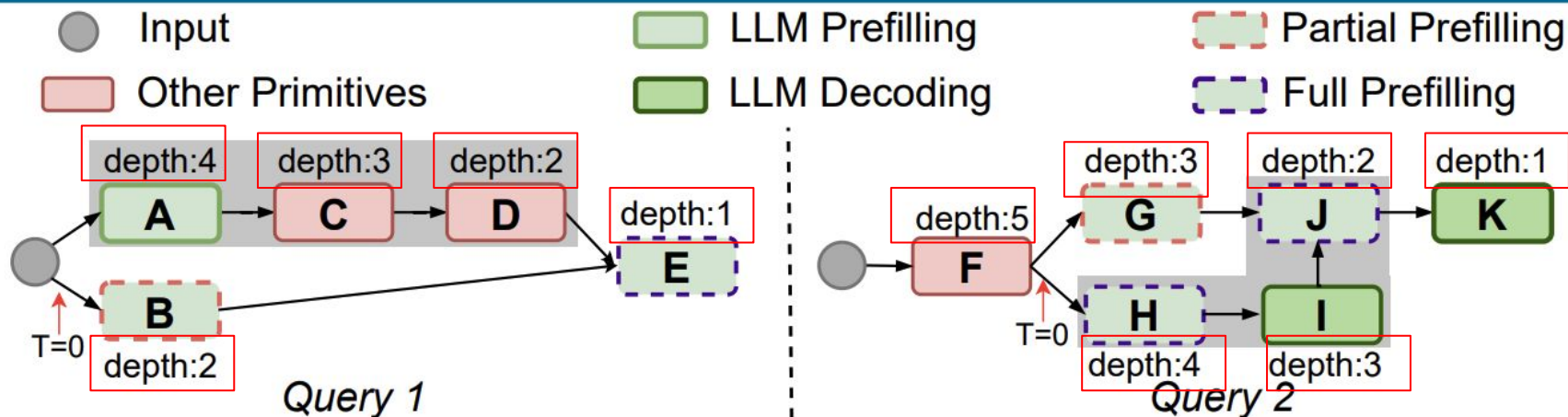


B is delayed. H goes before G.

Runtime Scheduling: Topology-aware batching



Runtime Scheduling: Topology-aware batching

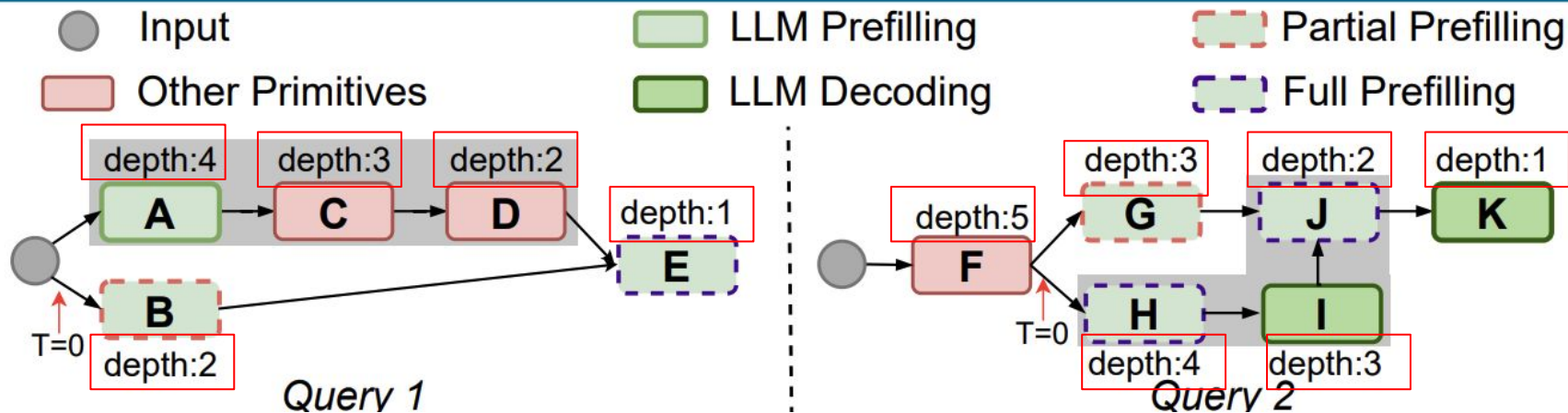


- **Reverse Topological Sorting:**

Assigns a "topological depth" to each primitive. Output nodes have depth 1; deeper upstream nodes have higher values.



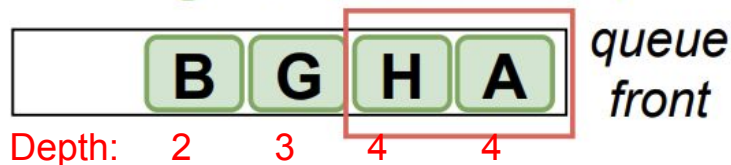
Runtime Scheduling: Topology-aware batching



- **Depth-Based Priority Queueing:**

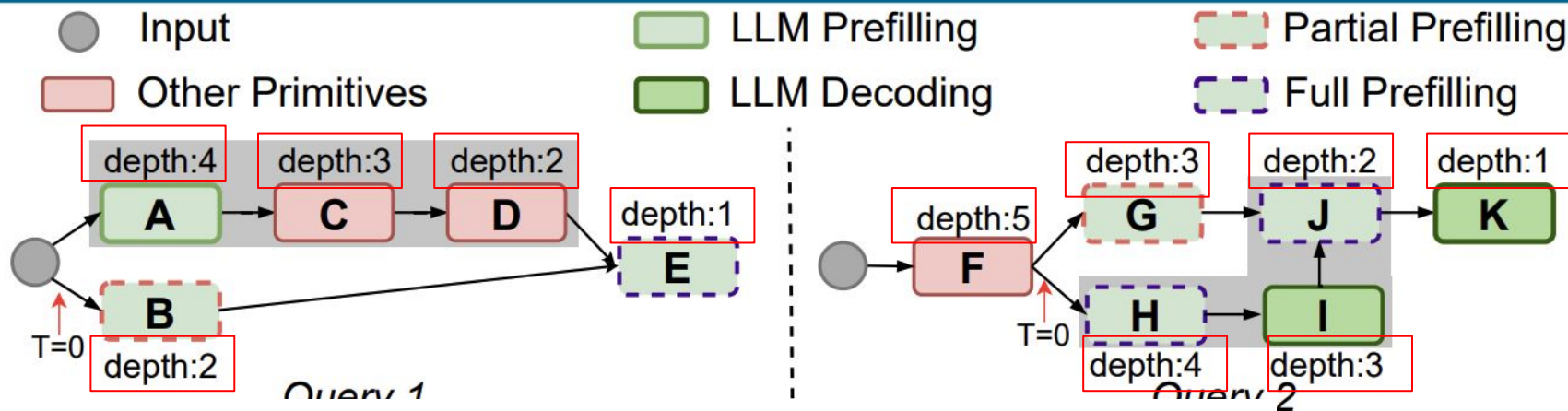
Tasks ready for execution are queued strictly by depth. Deeper primitives to completion (higher depth values) receive absolute priority.

LLM Engine (scenario 2)



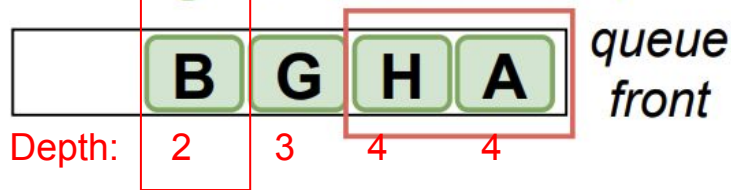
Note: the LLM engine only takes LLM primitives (green), not "other primitives" (red)

Runtime Scheduling: Topology-aware batching



- **Strategic Delay (Cross-Request Batching):**
Ayo intentionally *delays* new, low-depth requests to ensure critical-path tasks (e.g., final Decoding) have exclusive engine access.

LLM Engine (scenario 2)



Evaluation & Conclusion

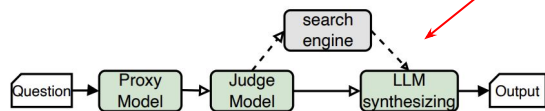
Experimental Setup

System Implementation:

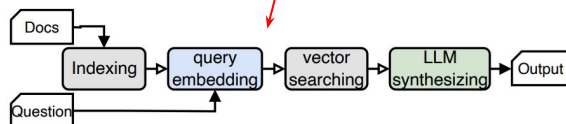
- Built upon Ray (for distributed scheduling), LlamaIndex, postgresql, pgvector, Google custom search, and vLLM.
- Models tested: gemma-2-2B, Llama-2-7B, Llama-30B.

Baselines & Workloads:

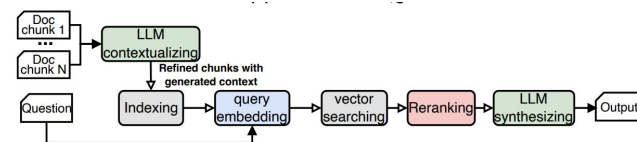
- **Baselines:** LlamaDist (Ray-based LlamaIndex), LlamaDistPC (with prefix caching), AutoGen (multi-agent framework).
- **Workloads:** Search Engine Gen, Naive RAG, Advanced RAG, Contextual Retrieval.



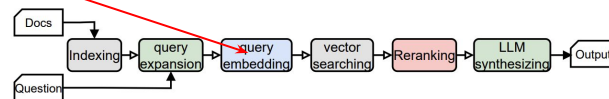
(a) Search engine-empowered generation



(c) Document QA with naive RAG

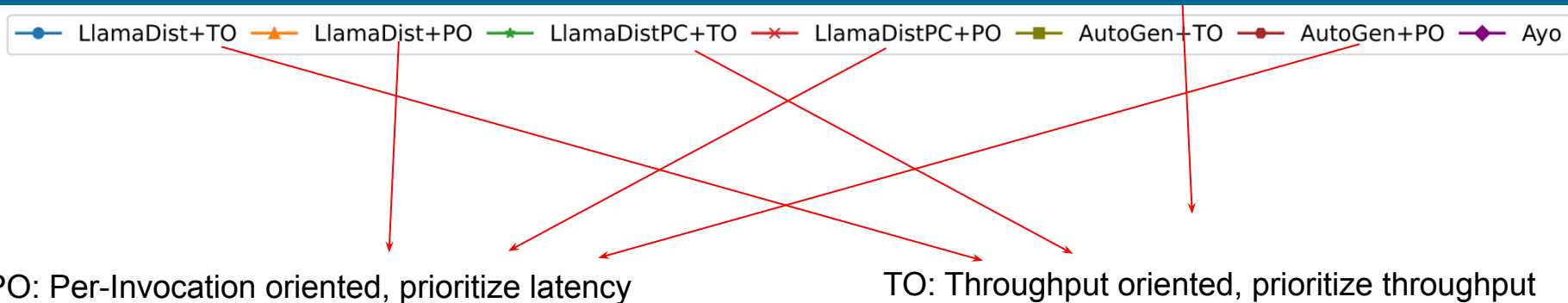


(e) Contextual retrieval from Anthropic [6]



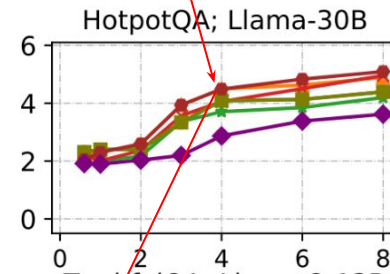
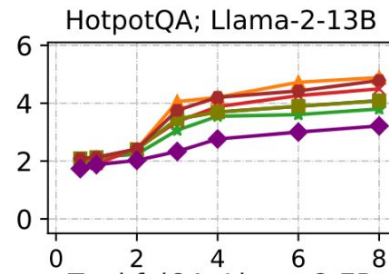
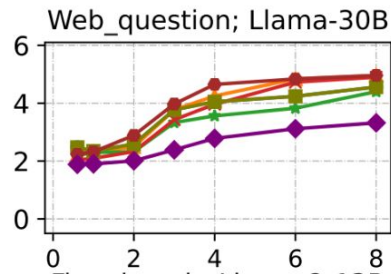
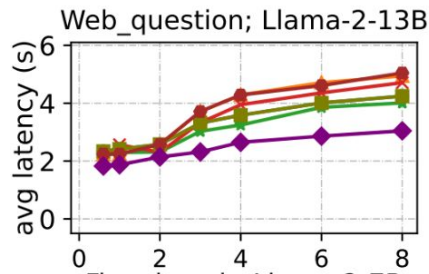
(d) Document QA with advanced RAG

Figure 8: End-to-end results

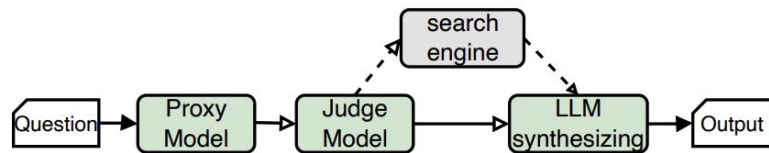


- LlamaDist
 - defines a chain of task modules to construct an application pipeline. Each task module invokes requests to different **distributed backend engines**. Utilizing the same engines as Ayo but differing in the **granularity** of orchestration
- LlamaDistPC (parallel & cache-reuse)
 - predefined pipeline and **manually parallelizes** independent modules for concurrent execution. It also incorporates **prefix caching** for LLM to avoid recomputation for partial instructions in the prompt
- AutoGen
 - **various agents** for an application, with each agent managing several system modules (e.g., a retrieval agent handling indexing, query embedding, and searching) and communicating with each other following a **pre-defined graph**.

—●— LlamaDist+TO
 —▲— LlamaDist+PO
 —★— LlamaDistPC+TO
 —✕— LlamaDistPC+PO
 —■— AutoGen+TO
 —●— AutoGen+PO
 —◆— Ayo



Insight: optimizing for **per-invocation latency** can backfire under load when the true objective is **end-to-end query completion time**.

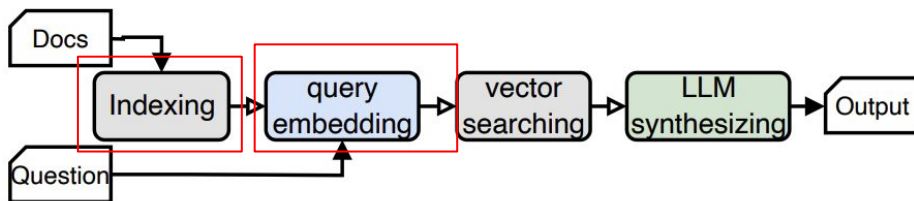
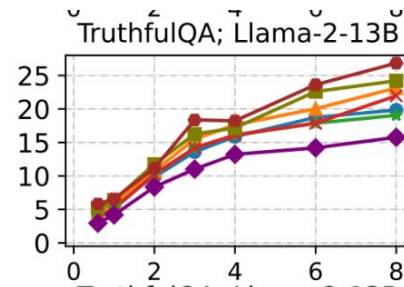
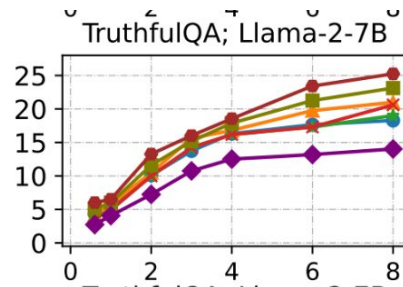
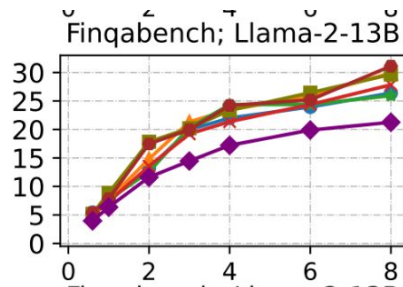
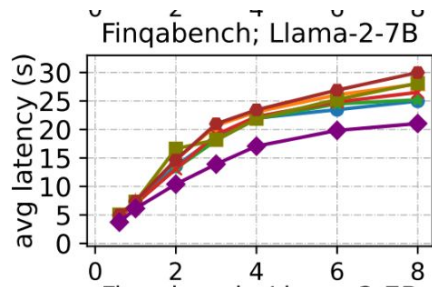


(a) Search engine-empowered generation

This workflow uses a **proxy/judge LLM** to decide whether search is needed, then optionally calls a search engine, and finally uses the **core LLM** to synthesize the answer.

Ayo beats the baselines by **up to 1.79×**.

—●— LlamaDist+TO
 —▲— LlamaDist+PO
 —★— LlamaDistPC+TO
 —✕— LlamaDistPC+PO
 —■— AutoGen+TO
 —●— AutoGen+PO
 —◆— Ayo



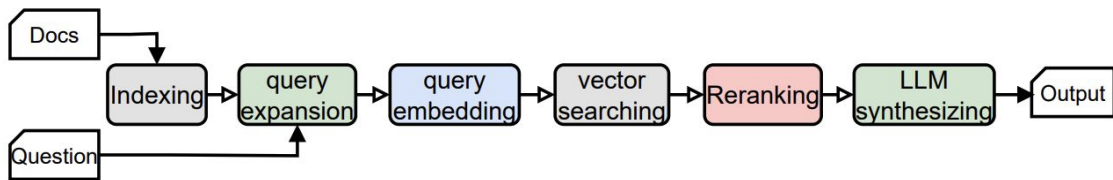
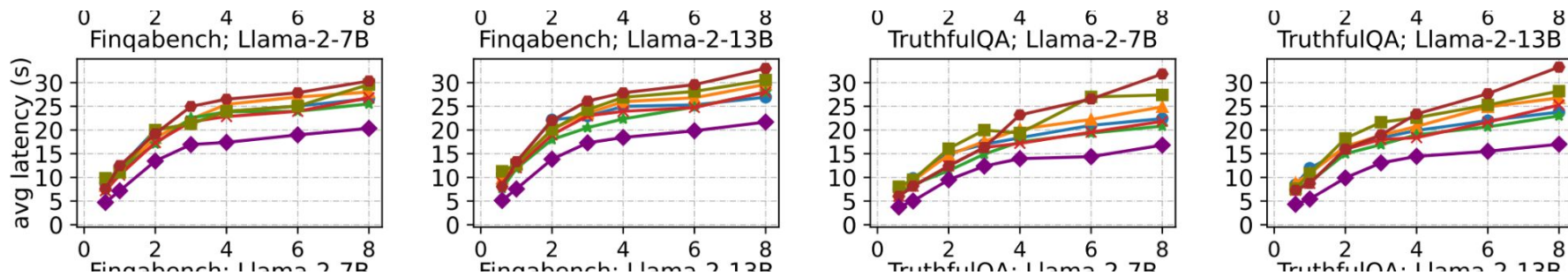
Insight: once an app has **multiple dependent subrequests hitting different engines**, **graph-aware scheduling** starts to matter a lot.

(c) Document QA with naive RAG

This workflow takes user documents or webpages, **chunks and embeds** them, stores them in a vector DB, retrieves the top chunks, and then uses the LLM to answer.

Ayo outperforms the baselines by **up to 1.62× at low request rates** and **1.67× at high request rates**.

● LlamaDist+TO
 ▲ LlamaDist+PO
 ★ LlamaDistPC+TO
 ✕ LlamaDistPC+PO
 ■ AutoGen+TO
 ● AutoGen+PO
 ◆ Ayo



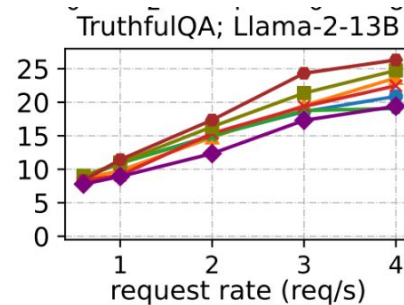
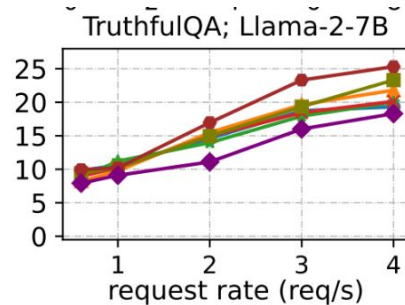
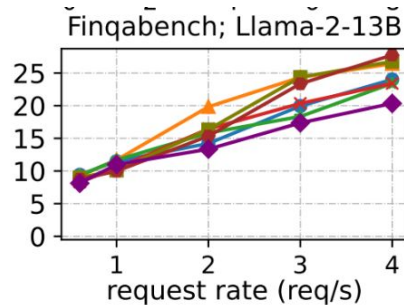
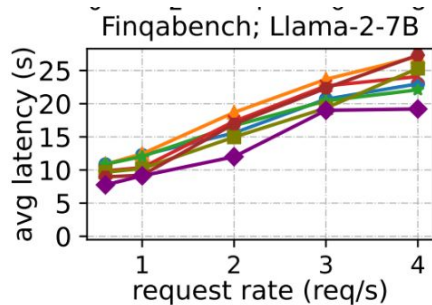
(d) Document QA with advanced RAG

It extends naive RAG with **query expansion**, **more retrieval**, **reranking**, and then **refine-mode synthesis**.

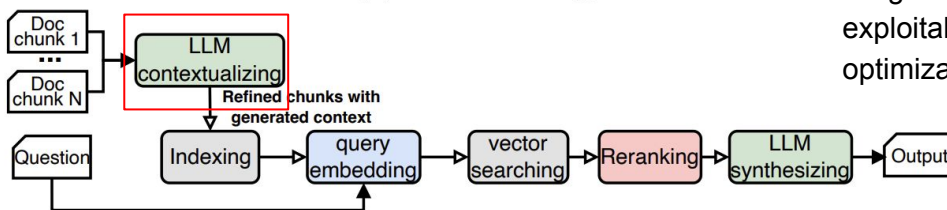
Ayo gets its biggest gains: **up to 2.09×** at low request rates and **2.03×** at high request rates.

Insight: the more complex and multi-stage the workflow, the more valuable **primitive-level end-to-end optimization** becomes.

—●— LlamaDist+TO —▲— LlamaDist+PO —★— LlamaDistPC+TO —✕— LlamaDistPC+PO —■— AutoGen+TO —●— AutoGen+PO —◆— Ayo



Insight: If one stage dominates and has little exploitable structure, then end-to-end graph optimization helps less.



(e) Contextual retrieval from Anthropic [6]

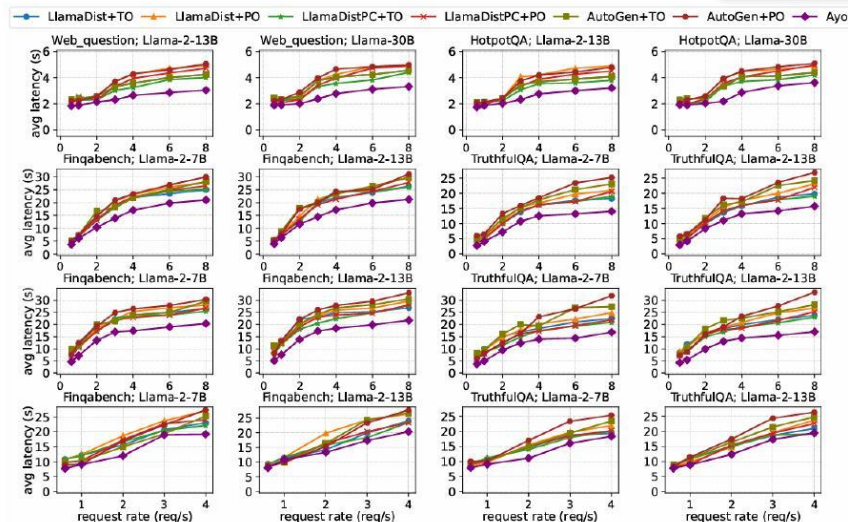
This workflow adds a contextualization step before indexing: a lightweight LLM summarizes each chunk using neighboring chunks, prepends that context, then retrieval and reranking proceed before the final LLM answer.

Ayo still wins by less: **about 1.06× to 1.59×** over baselines. The reason is that **contextualization dominates the total time** because there are so many chunks to process.

Key Results: E2E Latency Reduction

Performance Gain

Ayo achieves up to 2.09x E2E latency speedup over state-of-the-art orchestration frameworks under various request rates.



Ayo's benefit scales with how much cross-stage structure the workflow exposes.

The more a pipeline mixes LLM calls, retrieval, embedding, reranking, and dependent subrequests, the more end-to-end primitive-level optimization pays off.

Figure 9: Co-located applications

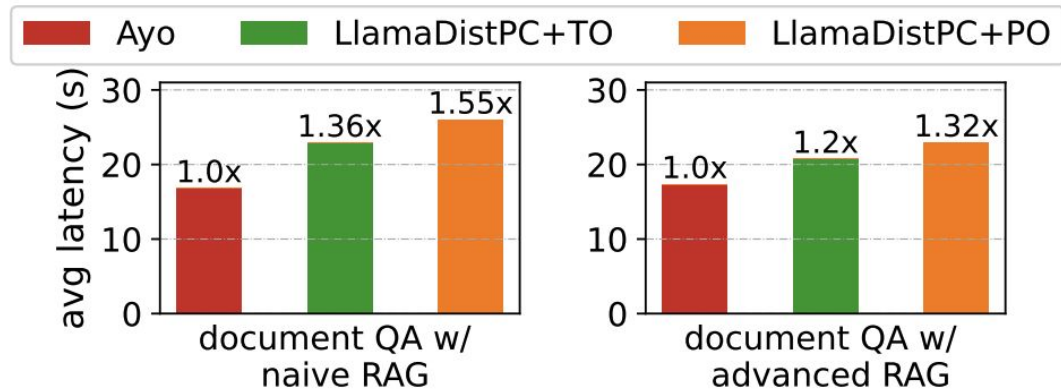


Figure 9. The average latency for two apps under a co-location scenario using llama-2-13B as the core LLM on truthfulQA dataset.

Figure 9 studies two applications running together on the same infrastructure: a naive RAG app and an advanced RAG app.

Ayo still improves latency by about **1.2x** to **1.55x** over the stronger baseline in this shared setting.

This is closer to **production reality**. In a real serving cluster, you rarely run only one app in isolation.

Figures 10–11: Ablations

Figure 10: graph optimization ablation

The paper disables parallelization or pipelining passes and shows both are useful. In advanced RAG, enabling both reduces latency for single-query runs and under load.

Figure 11: runtime scheduling ablation

Topology-aware batching improves average single-query execution by about **1.15×** according to the text.

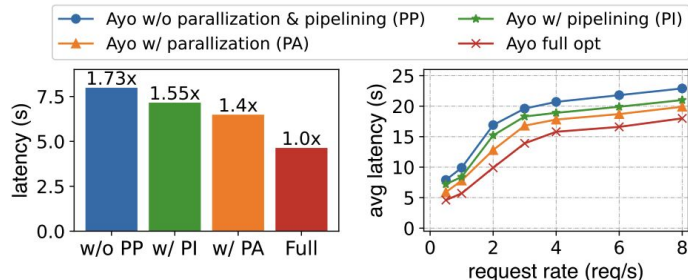


Figure 10. Ablation study on graph optimization in document QA with advanced RAG on truthfulQA dataset using llama-30B as core LLM. Left: single-query latency averaged over 10 runs. Right: average latency under varying request loads.

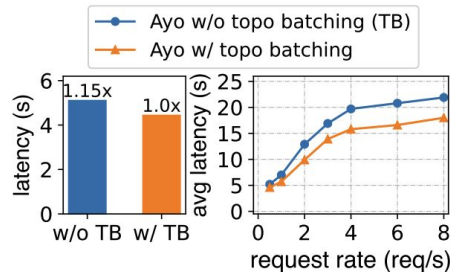


Figure 11. Ablation study on runtime scheduling. The setting is same as that in Figure 10.

Overheads

Critical-path latency breakdown

This figure analyzes where time is going in Teola's critical path. The text says:

- graph optimization overhead is only about **1.3%–3%**
- communication overhead is about **3.1%–6.2%**
- decomposed prefilling is only **3.11%–12.12%** slower than a monolithic prefill in microbenchmarks, and the extra parallelism more than compensates at the workflow level.

This is important because one might worry that “splitting the graph” adds too much orchestration overhead. The paper argues that the overhead exists but is small relative to the end-to-end gains

Partial Prefilling	Full Prefilling	Total	Single Prefilling
76.03 (200)	215.89 (800)	291.92 (1000)	260.36 (1000)
217.67 (850)	222.66 (850)	440.33 (1700)	414.09 (1700)
582.95 (2500)	159.65 (500)	742.60 (3000)	720.15 (3000)

Table 3. Execution efficiency comparison between decomposed prefilling (partial and full prefilling) of Ayo (left) vs. single complete prefilling (right) across various input sizes using Llama-2-7B. Values represent execution time in milliseconds, with input size shown in parentheses.

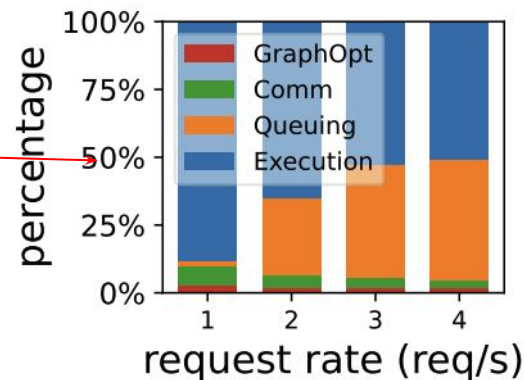


Figure 12. Latency breakdown of Ayo's execution critical path.

- Ayo: how do I optimize the whole application graph?
- Pie: how do I make the serving layer programmable for new application behaviors?
- Parrot: how do I exploit semantic relationships across multiple LLM calls?

Limitations

Ayo is best suited for **structured workflows with known templates**.

- It is less naturally suited to highly **dynamic autonomous** agents where the graph is not known ahead of time, especially self-reflective or open-ended planning loops.
- It also requires **tighter coupling** with backend engines than black-box orchestration frameworks do.
- The paper also points to future work around exploiting **critical-path** information for resource allocation and priority scheduling.

Conclusion

Summary of Contributions:

- Identified coarse-grained module orchestration as the primary bottleneck for E2E latency in LLM applications.
- Proposed the **Primitive-level Dataflow Graph (p-Graph)** to explicitly expose and optimize internal dependencies.
- Developed the **Ayo System**, featuring a Graph Optimizer and a Topology-Aware Runtime Scheduler.
- Demonstrated robust performance gains ($> 2\times$ speedup) across diverse, real-world LLM workloads.

Thank You!

AVA: Towards Agentic Video Analytics with Vision Language Models

Yuxuan Yan, Shiqi Jiang *et al.* — Microsoft Research & ZJU

Presented by: Xiangchen Song, Yicheng Tao

CSE 585 — University of Michigan

March 21, 2026

Outline

- 1 §1 Introduction
- 2 §2 Related Work and Motivation
- 3 §3 AVA System Overview
- 4 §4 Near-Real-Time Index Construction
- 5 §5 Agentic Retrieval and Generation
- 6 §6 Implementation
- 7 §7 Evaluation
- 8 §8 Conclusion
- 9 Discussion

§1 Introduction — Problem & Vision

Real-world video analysis demands:

- Videos spanning **hours to days**
- **Open-ended** NL queries — no predefined tasks
- **Near real-time** processing on edge servers

Current systems (L1–L3): Closed-ended, task-specific. Cannot answer arbitrary questions.

AVA's goal: First **L4** system — open-ended understanding, reasoning, and analytics. *Note: L1–L5 proposed by authors (like SAE driving levels).*

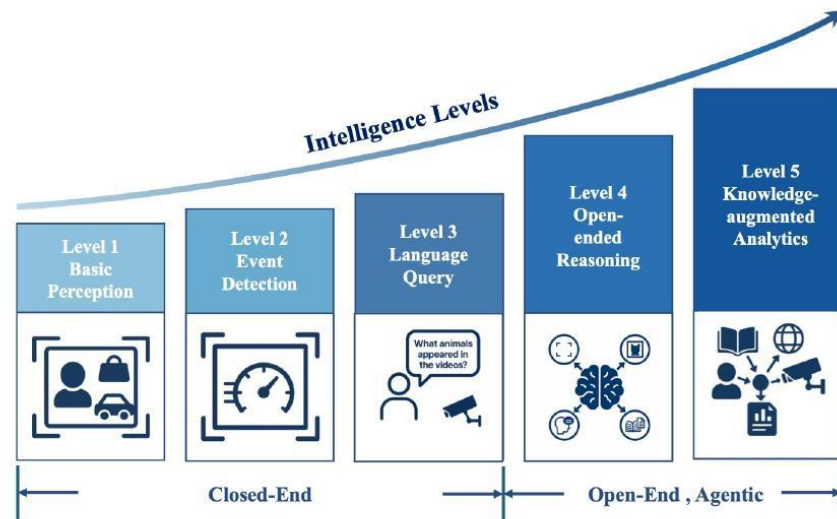


Figure 1: Intelligence levels of video analytics systems.

§1 Introduction — Running Example

Running Example

Imagine a **10-hour mall surveillance video**. A user asks:

“What did the woman in red buy? What did she do before purchasing?”

This requires: (1) identifying a specific person across hours of footage, (2) finding the purchase event, (3) tracing her prior activities. **No existing system can do this.** We use this example throughout.

§2 Related Work and Motivation — Three Existing Paradigms

Paradigm 1: Uniform Sampling

Sample frames at fixed rate, feed all into VLM.

Problem: Context $\leq 1\text{M}$ tokens $\Rightarrow$ max $\sim 1\text{h}$. $10\text{h} \rightarrow$ sampling drops $10\times$.

Paradigm 2: Video RAG

Embed frames with CLIP, retrieve top- k by similarity, feed to VLM.

Problem: Flat retrieval — no structure for temporal reasoning.

Paradigm 3: KG-RAG

Build KG to enhance retrieval (GraphRAG, LightRAG).

Problem: All existing KG-RAG is **text-only**.

Gap: No system combines structured KG indexing + agentic retrieval for *video*.

§2 Related Work and Motivation — Key Experiment: Few Frames Suffice

Experiment on VideoMME with Qwen2-VL:

- 1 Find questions answerable at 1 FPS
- 2 **Binary search** for minimum frames needed
- 3 Result: only a **handful of frames** suffice

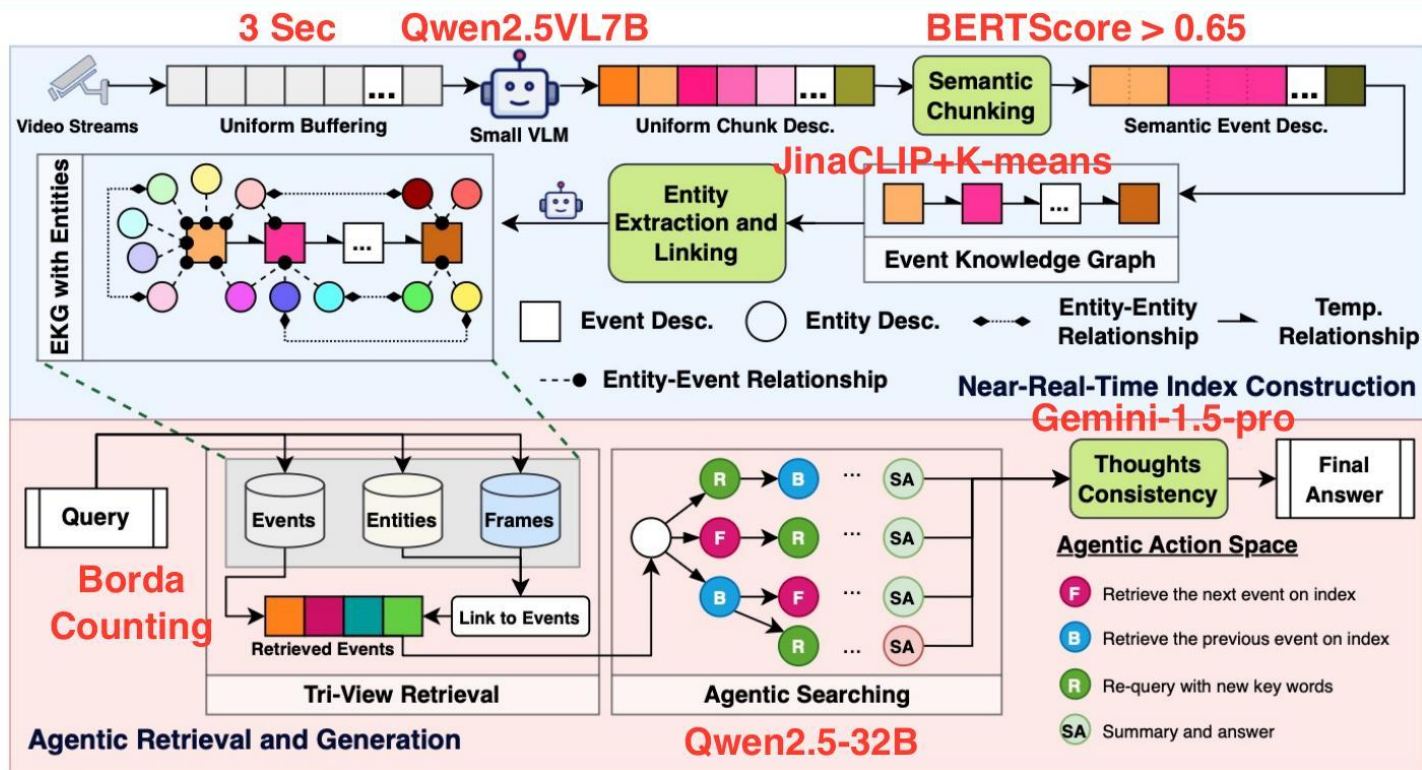
Insight: Build an intelligent index to *pinpoint* the few relevant frames. **VLM size dilemma:** Large (72B) too slow; Small (7B) degrades on long input.

⇒ **Semantic chunking** lets small VLMs work well.

Short (1.4 minutes)		Medium (9.7 minutes)		Long (39.7 minutes)	
Total	Needed	Total	Needed	Total	Needed
2144.8	12.1 (0.5%)	13924.1	68.1 (0.4%)	66847.1	82.3 (0.1%)

Table 1: Only a small portion of the frames are necessary to answer each particular question across the short, medium, and long video subsets of VideoMME [14] benchmark using Qwen2-VL.

§3 AVA System Overview



Design Principles: (1) Scalable to any video length (2) Near real-time indexing (>1 FPS) (3) Handle diverse, complex queries

Top half: Near-Real-Time Index Construction (§4) — Video stream → EKG

Bottom half: Agentic Retrieval & Generation (§5) —

Query + EKG → Answer

§4.1 Event Knowledge Graph (EKG)

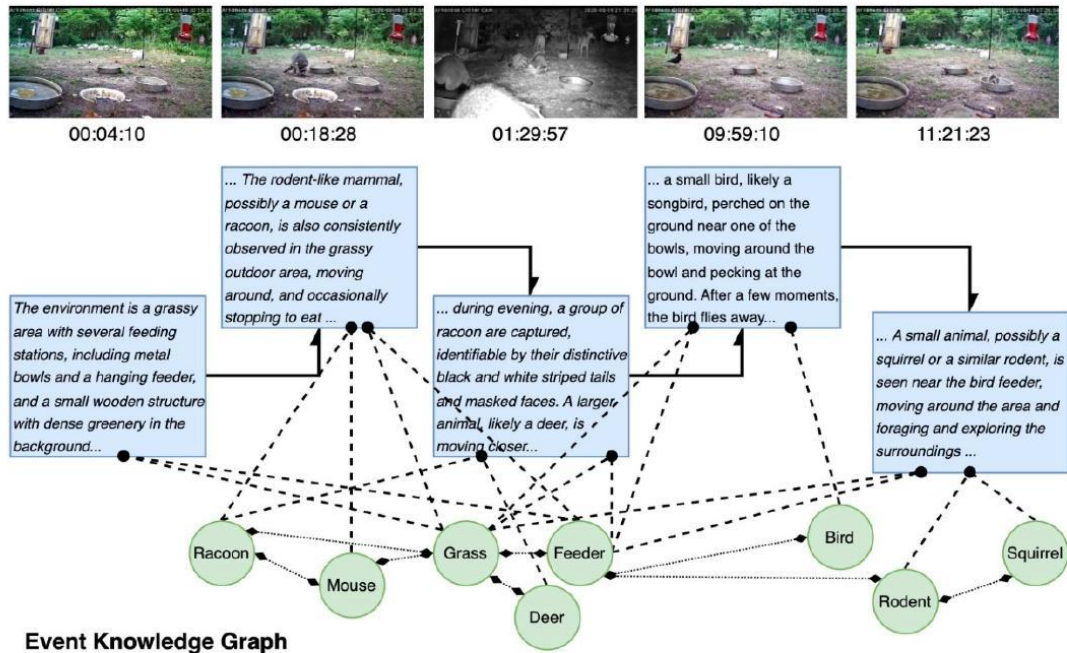
Definition: $G = (E, U, R)$

- E : temporally ordered **events**
- U : **entities** per event
- $R = R_{ee} \cup R_{uu} \cup R_{ue}$
 - R_{ee} : event $\rightarrow$ event (**temporal**)
 - R_{uu} : entity $\leftrightarrow$ entity (**semantic**)
 - R_{ue} : entity $\leftrightarrow$ event (**participation**)

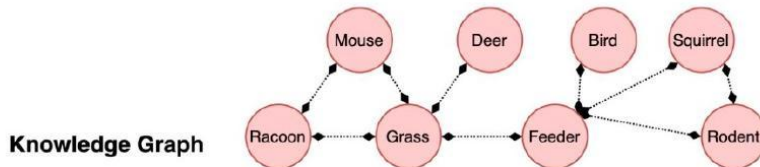
EKG vs. Traditional KG

KG (bottom): static triples, no time

EKG (top): events with timestamps; entities linked via participation



Event Knowledge Graph



Knowledge Graph

§4.2 Semantic Chunking (Figure 4)

Problem: Uniform 3s chunks break event coherence.

Solution: Merge similar adjacent chunks.

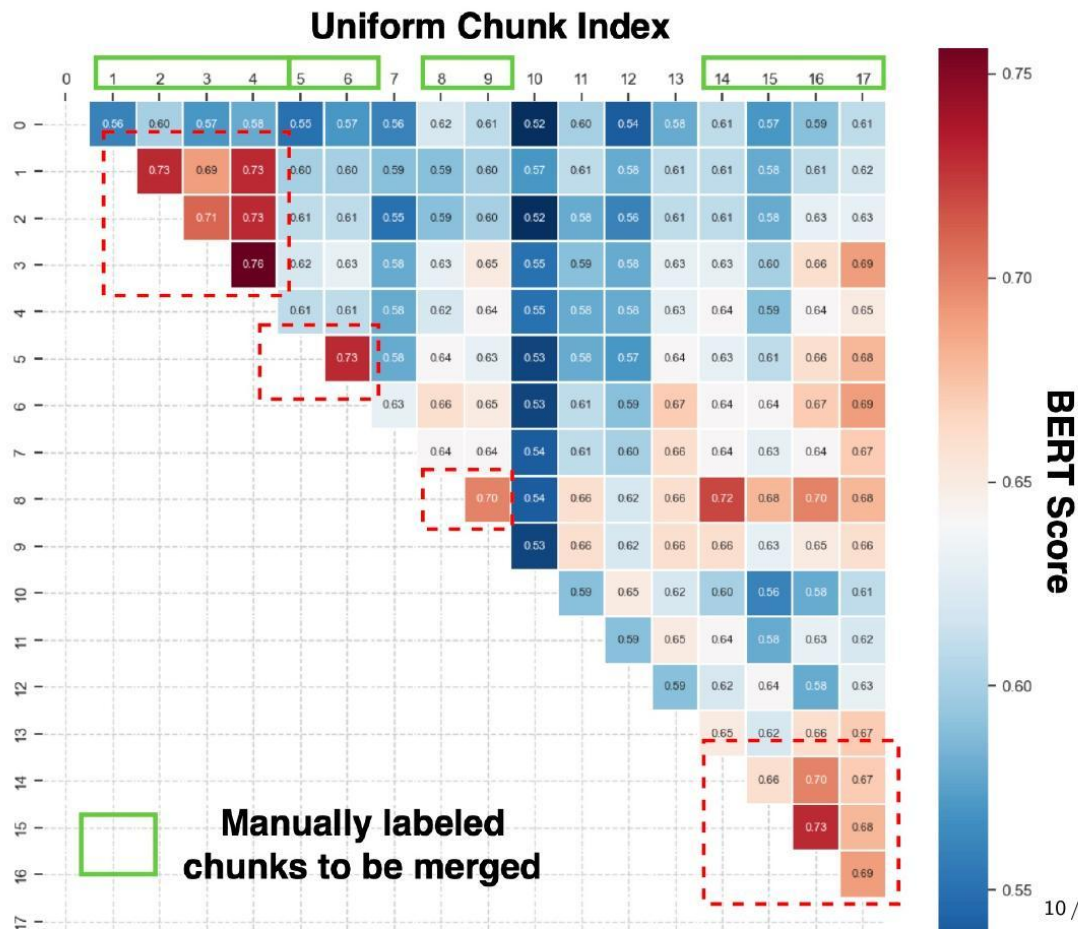
4 steps:

- 1 **Uniform buffering:** 3s chunks
- 2 **VLM description:** Qwen2.5-VL-7B
- 3 **BERTScore:** adjacent pair similarity
- 4 **Merge / Cut:** $BS \geq 0.65$ merge; else cut

Result: 18 uniform $\rightarrow$ 9 semantic chunks.

Red dashed = high BERTScore clusters.

Green = human-annotated merge groups.



§4.3 Entity Extraction, Linking & Pipeline

Entity extraction: Small VLM extracts entities + relations per semantic chunk.

De-duplication:

e_1 : “woman in red coat” vs. e_3 : “female customer paying” — exact match **fails**.

Solution: JinaCLIP embedding + K-means clustering
→ unified entity.

JinaCLIP: multimodal model, same vector space for text & images.

EKG Database

Table	Content
events	ID, time, description
entities	ID, desc, centroid emb.
$R_{ee}/R_{uu}/R_{ue}$	temporal/semantic/participation
frame index	JinaCLIP → events

Near-Real-Time Pipeline

All stages in **parallel**: VLM describes c_i while BERTScore runs on c_{i-2} . EKG grows incrementally. >5 FPS on 2×GPU edge.

§5.1 Tri-View Retrieval (Figure 5)

Three retrieval views:

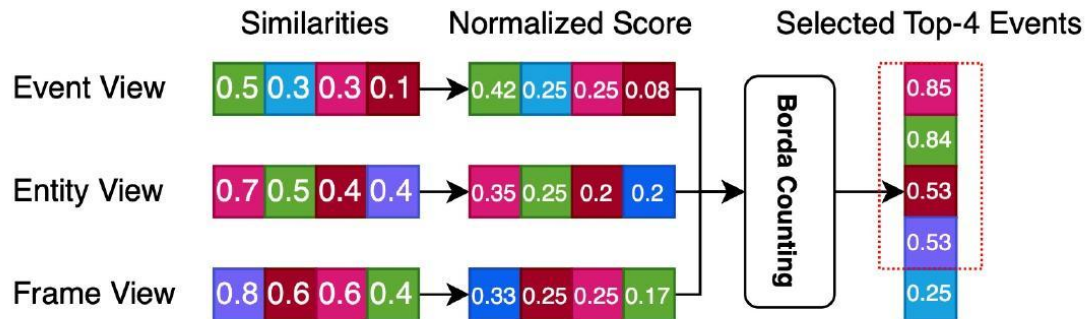
- 1 **Event:** query $\leftrightarrow$ event desc.
- 2 **Entity:** query $\rightarrow R_{ue} \rightarrow$ events
- 3 **Frame:** query $\leftrightarrow$ frame emb.

Borda counting:

$$s_m(e_j) = \frac{\text{sim}_m(e_j)}{\sum_{e_k} \text{sim}_m(e_k)}$$

$$s(e_j) = \sum_m s_m(e_j)$$

Normalizes, then sums.



Running Example

Query: "What did the woman in red buy?"

Event $\rightarrow e_4, e_3$ **Entity** $\rightarrow e_1-e_4$ **Frame** $\rightarrow$ counter

Borda: $e_4 > e_3 > e_2 > e_1$

§5.2 Agentic Searching on Graph (Figure 6)

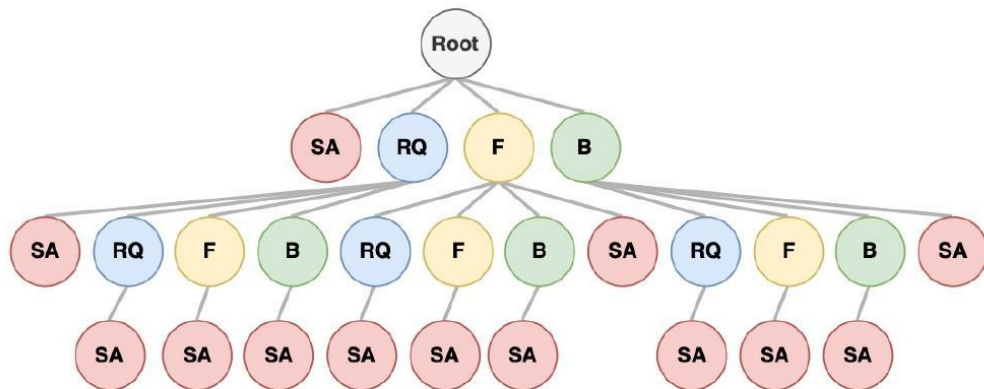
4 agent actions:

- **F**: next event via R_{ee}
- **B**: previous event via R_{ee}
- **RQ**: re-query, new keywords
- **SA**: stop, generate answer

Max depth = 3. ~13 pathways.
Agent = Qwen2.5-32B.

vs. flat retrieval:

Flat finds e_4 . “What before?” needs **B**:

$$e_4 \rightarrow e_3 \rightarrow e_2 \rightarrow e_1.$$


Each leaf SA node generates a candidate answer.

§5.3 Consistency-Enhanced Generation + Check Frames and Answer

Multi-generation per SA node:

CoT, n times $\rightarrow$ (thought r_i , answer a_i)

Answer agreement: $S_a^{(t)} = \frac{|\{i | a_i = a^{(t)}\}|}{n}$

Thought consistency: $S_r^{(t)} = \frac{2}{k(k-1)} \sum_{i < j} \text{BS}(r_i, r_j)$

Final: $S_{\text{final}}^{(t)} = \lambda S_a^{(t)} + (1 - \lambda) S_r^{(t)}$

8 iterations, $\lambda \approx 0.3$.

Check Frames & Answer (CA)

Trigger: top-2 SA nodes differ.

Action: Raw frames $\rightarrow$ **Gemini-1.5-Pro**.

Why? EKG misses details ("an item" vs. *Dior perfume*).

Progressive Model Escalation

Index (§4): Qwen2.5-VL-**7B** (cheap)

Agent SA (§5.2): Qwen2.5-**32B** (text only)

CA (§5.3): Gemini-1.5-**Pro** (visual)

Expensive model only when needed.

§6 Implementation

Acceleration stack:

- **AWQ** (INT4): $\sim 4\times$ compression
- **LMDeploy**: KV cache (30% GPU)
- Pipeline parallelism for indexing
- Frames pre-extracted to disk

Index speed: >5 FPS on $2\times$ GPU edge.
10h video $\rightarrow$ 3–5h to build EKG.

Hyperparams: BERTScore 0.65 / Depth 3 / Self-consistency 8

Query Latency — Table 2 ($1\times$ A100)

Stage	Latency	VRAM
Tri-View Retrieval	0.44s	<1 GB
Agentic Search (14B)	101.5s	—
Agentic Search (32B)	174.2s	—
CA: Qwen-VL-7B	45.8s	~ 31 GB
CA: Gemini API	lower	0

Bottleneck: Agentic search — LLM call at every tree node. Motivates **learned policies**.

§7 Evaluation — Main Results (Figure 7)

Benchmarks:

LVBench: 103 videos, 68min, 1549 Qs

VideoMME-L: 300 videos, 40min, 900 Qs

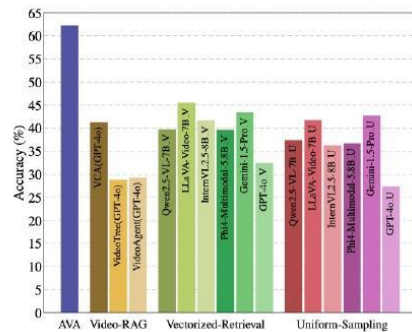
AVA-100: $8 \times 10+h$, 120 Qs

+16.9% LVBench

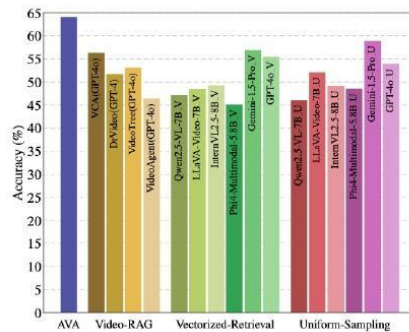
+5.2% VideoMME-L

+20.8% AVA-100 **Notable:**

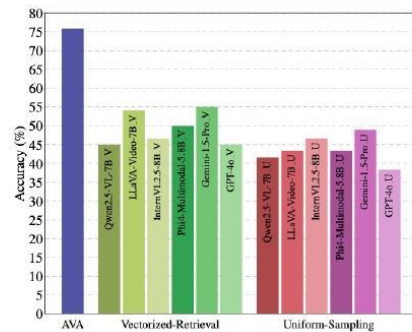
Text-only AVA beats VLMs that see frames.



(a) LVBench



(b) VideoMME-Long



(c) AVA-100

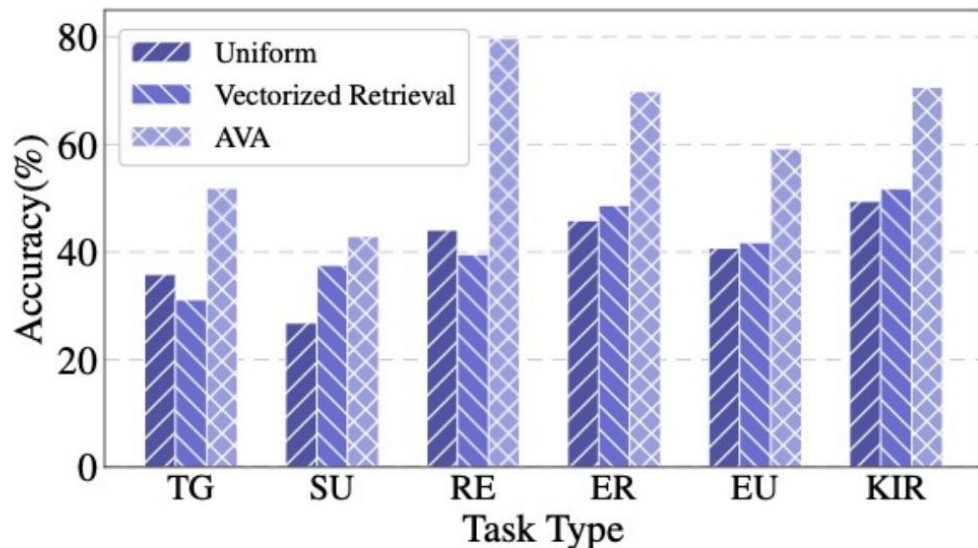
§7 Evaluation — Per-Task Breakdown (Figure 8)

Gains over best baseline (LVBench):

Task Type	Gain
Temporal Grounding	+16.0%
Summarization	+5.3%
Reasoning	+35.6%
Entity Recognition	+21.2%
Event Understanding	+17.5%
Key Info Retrieval	+18.9%

Largest: Reasoning — causal chains.

Smallest: Summarization — global coverage.



§7 Evaluation — Ablation: EKG vs. KG & Search Depth

Ablation 1: EKG vs. Traditional KG (Table 3)

Method	Acc.	Overhead(h)
MiniRAG	28.1	3.49
LightRAG	30.6	3.52
AVA	39.7	0.31

EKG: +9% accuracy, 11× faster build.

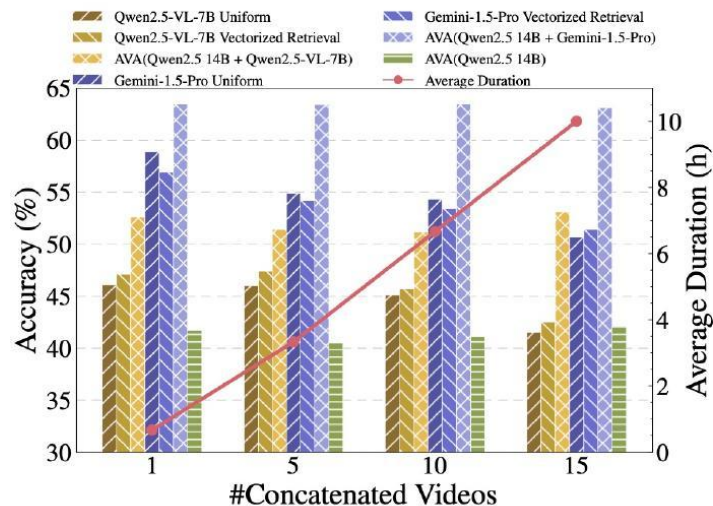
Ablation 2: Tree Search Depth (Table 4)

Method	Tree Search Depth			
	1	2	3	4
AVA(Qwen2.5 14B)	34.1	36.1	40.9	39.5
AVA(Qwen2.5 14B + Qwen2.5VL 7B)	49.3	52.1	53.8	50.2
AVA(Qwen2.5 14B + Gemini-1.5-Pro)	54.2	58.4	61.5	52.7
Tree Search Overhead(s)	6.7	27.3	90.1	370.3

Depth 3: best accuracy. Depth 4: drops, 4× overhead.

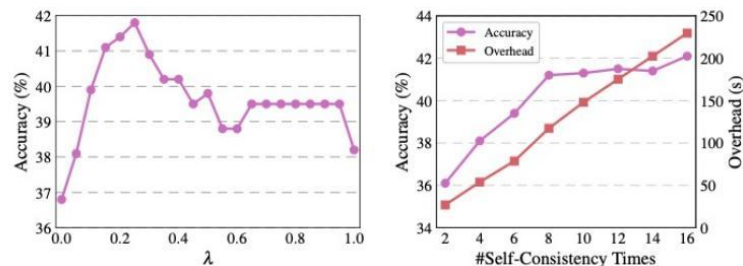
§7 Evaluation — Robustness (Fig. 10) & Consistency (Fig. 12)

Robustness to Video Length



Baselines drop 4–8% at 10h. **AVA** stays flat.

Consistency Settings



(a) Balance between thoughts and answer consistency (b) Trade-offs using different self-consistency times

(a) $\lambda \approx 0.3$ best. (b) 8→16: +0.9% but 2× cost.

§8 Conclusion — Limitations

Limitation 1: Fixed Tree Search

- Fixed depth/branching for all queries
- **Latency bottleneck** (101–174s)

Fix: Learn a policy from trajectories that **dynamically** selects actions & depths.

⇒ *RL-guided search*.

Limitation 2: VLM on Specialized Tasks

- Object counting, fine-grained recognition
- Spatial reasoning

Fix: Integrate tools (YOLO, OCR), let agent decide *when* to invoke.

⇒ *Tool-augmented agent* ($L4 \rightarrow L5$).

§8 Conclusion — Future Research Directions

My Proposed Directions

- 1 **RL-learned search policy:** Replace fixed tree search with learned agent. AVA's trajectories = natural offline RL data.
- 2 **KV-cache sharing:** Many LLM calls with overlapping context — prefix parallelism could cut latency.
- 3 **Audio modality:** EKG is vision-only. ASR could help dialogue-heavy videos.

Discussion — Strengths & Weaknesses

Strengths

- 1 **Novel EKG:** Temporal event-centric KG for video
- 2 **Scalable:** Decoupled from video length
- 3 **Smart allocation:** 7B/32B/Gemini escalation
- 4 **SOTA** on 3 benchmarks with thorough ablations
- 5 **AVA-100:** New ultra-long benchmark
- 6 **Honest limitations** with clear future work

Weaknesses

- 1 **Fixed search:** Same depth for all queries
- 2 **High latency:** 101–174s; not interactive
- 3 **BERTScore 0.65:** Generalization unclear
- 4 **K-means:** Needs k ; unbounded vocab
- 5 **No audio:** Misses dialogue, narration
- 6 **Proprietary CA:** Gemini now deprecated

Comparison

System	Research Gap	Intuition behind Solution	Limitations
Pie (ASPLOS '24)	The Orchestration Tax: Existing engines treat LLMs as black boxes, causing massive data movement and latency when complex logic (loops/if-else) sits outside the engine.	Logic-to-Data: Bring the application logic <i>into</i> the serving system. By making the engine "programmable," you can manage KV-caches and scheduling much more precisely.	Developer Friction: Requires developers to rewrite their apps using Pie's specific API; potential security risks of running user code in the engine.
Parrot (OSDI '24)	Redundant Computation: Chained LLM calls (like RAG or agents) often repeat the same context, but current systems don't realize these requests are semantically linked.	Semantic Variables: Treat inputs/outputs as variables rather than raw strings. This "Call-by-Semantic" approach allows for aggressive KV-cache sharing across different requests.	Predictability Dependent: If an application's flow is highly random or non-repetitive, the benefits of semantic speculation and variable reuse diminish.
Ayo (2024/25)	Local vs. Global Optima: Developers tune prompts, pick models, and set retrieval parameters in isolation, missing the "big picture" performance/cost trade-offs.	App-as-a-Graph: View the LLM app as a computation graph. Use a compiler-like optimizer to automatically find the best "hyperparameters" (e.g., prompt length vs. model size) globally.	Optimization Complexity: The search space for a global optimum is huge; the system relies heavily on the accuracy of its internal "cost models" to make decisions.
AVA (Agentic Video Analytics)	VLM Cost/Latency: Using Vision-Language Models (VLMs) for video is prohibitively slow and expensive; traditional CV (object detection) is too "dumb" for complex queries.	Agentic Filtering: Use a "manager" agent to dynamically decide when to use a cheap, fast model (like YOLO) and when to trigger an expensive VLM only for "hard" frames.	Latency Jitter: Because the agent decides when to "think harder," processing time per frame can vary wildly, making real-time guarantees difficult.

Key Takeaways

- ① **Structure over brute-force:** EKG outperforms directly feeding frames to VLMs.
- ② **Agentic search for complex reasoning:** Multi-step graph exploration beats flat retrieval for temporal/causal queries.
- ③ **Smart allocation:** Small models for indexing, large only when needed.
- ④ **Open problem:** Fixed tree search is the bottleneck. RL-based adaptive policies are promising.

Thank you! Questions?